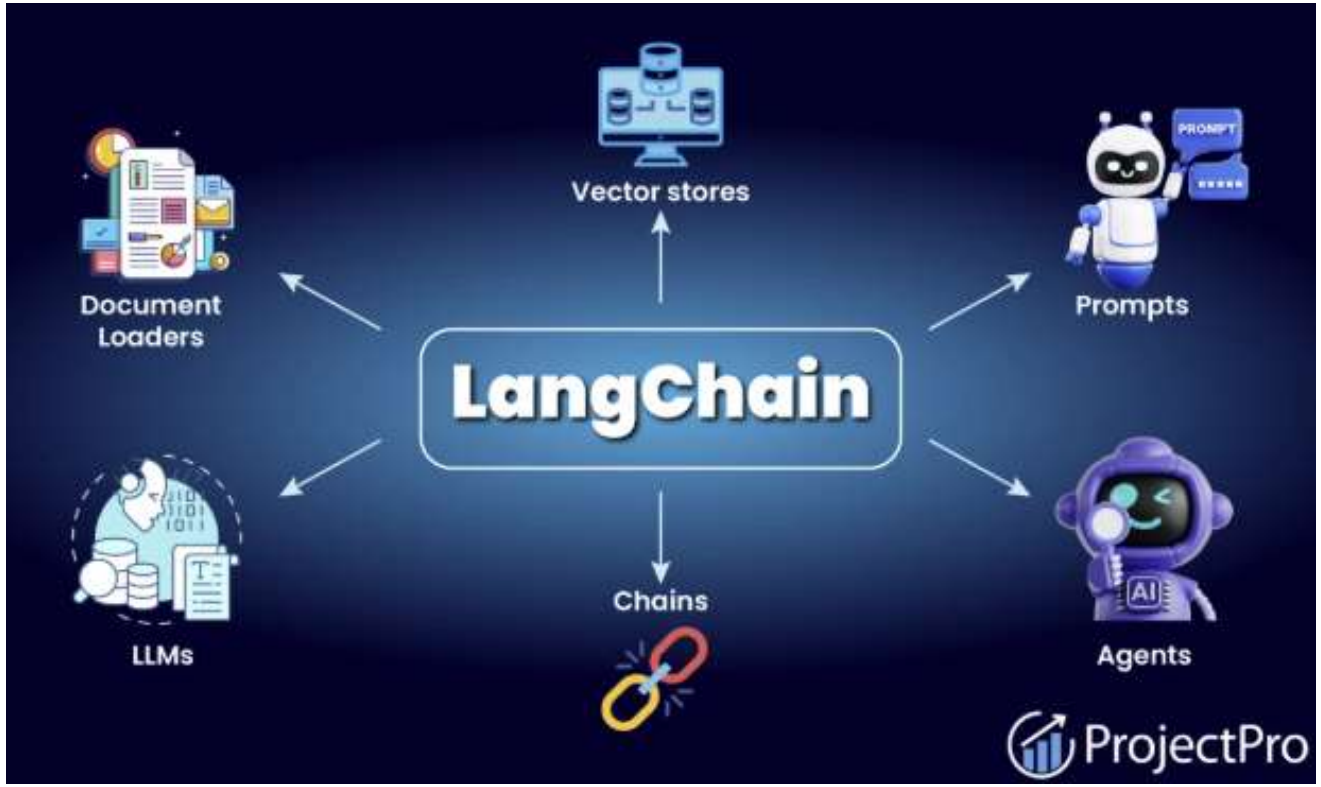
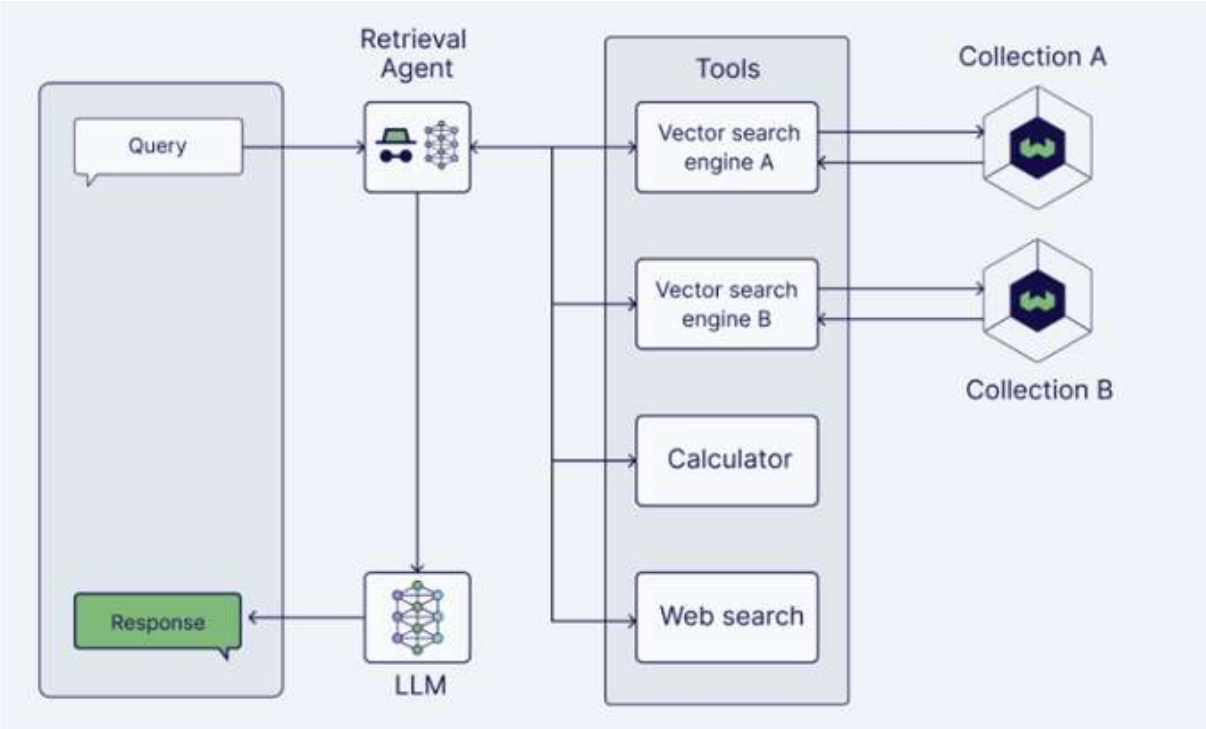


LangChain.js

- LangChain 소개
- Preview : Agent 사용 ChatBot 실습
- Message 클래스
- Model 클래스
- Prompt
- Chain
- Output Parser
- RAG
- Tool
- 메모리와 대화 관리
- Agent



LangChain 이란

LangChain(Python) vs LangChain.js

LangChain(Python)	LangChain.js(JavaScript/TypeScript)
• 복잡한 에이전트, multi-step reasoning, tool calling, custom memory, RAG + re-ranking 등 고급 기능이 필요할 때 • 최대한 많은 LLM, 벡터 DB, 툴을 한 번에 테스트하고 싶을 때 • 장기 유지보수 & 안정성이 중요한 프로덕션 프로젝트 • 이미 팀에 Python 개발자가 많거나 데이터/AI/ML 경험이 풍부할 때 • LangGraph 기반의 stateful 그래프 워크플로우를 본격적으로 만들 계획일 때	• 프론트엔드/엣지에서 LLM을 직접 호출하고 싶을 때 (Next.js App Router + Vercel AI SDK 조합 등) • Cloudflare Workers, Deno, Bun 같은 매우 가벼운 환경에서 동작시켜야 할 때 • 이미 팀이 TypeScript Full Stack 으로 움직이고 있고, Python 개발자를 추가 채용하기 어려운 상황 • 간단한 채팅봇, RAG 챗, tool calling 1~2개 정도의 중간 이하 복잡도 애플리케이션 • 배포 속도와 Front-BackEnd코드 공유를 극대화하고 싶을 때

LangChain 이란

LangChain

- 해리슨 체이스(Harrison Chase)에 의해 2022년 10월에 오픈 소스 프로젝트로 시작
- 챗봇, 질의응답 시스템, 자동 요약 등 다양한 애플케이션과 파이프라인을 신속하게 구축할 수 있는 플랫폼
- OpenAI, Cohere, Hugging Face 등과 같은 다양한 LLM Provider 업체로부터 모델을 사용할 수 있게 하는 플랫폼
- 다양한 LLM 제공 업체 간의 호환성을 보장하며, 사용자는 복잡한 API 변환 작업 없이 여러 LLM을 쉽게 탐색하고 사용 할 수 있다
- 동기(sync), 비동기(async), 배치(batching), 스트리밍(streaming) 모드에서 모델을 사용할 수 있는 기능을 제공
- 2024년 12월 LangChain 1.0을 정식 출시

langchain 1.0부터 주요 변화점 :
createAgent 라는 새로운 표준 에이전트 생성 방식 도입
미들웨어 기반 customizing 강화
표준 콘텐츠 블록 지원 확대
패키지 구조 정리 (구형 기능은 @langchain/classic으로 이동)
공식 문서: https://docs.langchain.com/oss/javascript/langchain/overview
마이그레이션 가이드: https://docs.langchain.com/oss/javascript/migrate/langchain-v1

LangChain 이란

LangChain

- langchain 1.0 - 5계층 아키텍처

계층	특성
1. AI 추론 및 생성 계층	다양한 LLM 모델(OpenAI, Anthropic, Google 등)을 표준화된 인터페이스로 사용 특정 공급자에 종속되지 않고 자유롭게 모델을 교체할 수 있습니다.
2. 외부 기능 계층	API, 데이터베이스, 웹 서비스 등 외부 시스템과 연동 Tool Calling을 통해 LLM이 정의된 함수를 직접 호출
3. 오케스트레이션 계층	LangGraph를 기반으로 하는 에이전트(Agent)를 통해 복잡한 추론과 행동의 흐름을 제어 내구성 있는 실행(durable execution), 스트리밍, 영속성을 지원
4. 컨텍스트 보존 계층	Message History, Custom State Schema, 장기 메모리를 통해 대화의 맥락을 유지 토큰 한계를 넘지 않으면서 효과적으로 컨텍스트를 관리
5. 정보 접근 계층	문서 Loader, 텍스트 분할기, 벡터 스토어, 검색기(Retriever)를 통해 RAG(Retrieval-Augmented Generation) 패턴을 구현

LangChain 이란

LangChain.js 중요 핵심 패키지

- 패키지 구조가 모듈화되어 있고, LCEL(LangChain Expression Language) 기반의 체인 구성, Agent, Tool, Memory 등을 중심으로 설계

패키지명	역할 및 설명
@langchain/core	LCEL 지원 : Runnable 기반 체인 구성 OpenAI, Anthropic 등 모델 과 분리, 독립된 구조 TypeScript 중심 pipe() 기반 데이터 흐름의 체인 구성 표준 Base LLM/ChatModel 인터페이스 Message 객체 (HumanMessage, AIMessage 등) Content block, tool call, standardized schemas, Callback / Tracing Runnable (LCEL), output parser 기본 클래스 등

```
npm install langchain
npm install @langchain/core
```

레거시 기능이 필요할 때 → 기존 체인, 대부분의 document loader/retriever/vectorstore 등

```
npm install @langchain/classic
```

LangChain 이란

LangChain.js 모델/통합 패키지

패키지명	역할 및 설명
@langchain/openai	OpenAI API (gpt-4o, gpt-4.1, o1 등) + Azure OpenAI 모델 지원 모델 : ChatOpenAI , 임베딩: OpenAIEmbeddings
@langchain/anthropic	Anthropic Claude 시리즈 모델 지원 (Claude 3.5, 3.7 등) 모델 : ChatAnthropic
@langchain/google-genai	Google Gemini 시리즈 (Gemini 1.5, 2.0 등) 모델 : ChatGoogleGenerativeAI 임베딩: GoogleGenerativeAIEmbeddings
@langchain/cohere	Cohere Command R / Aya 등의 모델 ChatCohere (채팅) CohereEmbeddings (임베딩)
@langchain/community	커뮤니티 기여 모델·툴·유틸리티 모음 다양한 서드파티 LLM, 임베딩, vector store, document loader 등 (예: Ollama, HuggingFace, Fireworks 등)

LangChain 이란

LangChain.js 중요 핵심 패키지

패키지명	역할 및 설명
langchain	에이전트 빌딩의 중심 패키지 createAgent factory ReAct 에이전트 등 prebuilt messages, prompts 일부 편의 re-export
@langchain/langgraph	
@langchain/pinecone	
@langchain/textsplitters	
@langchain/document-loaders	

```
npm install @langchain/anthropic
npm install @langchain/google-genai
npm install @langchain/anthropic
npm install @langchain/cohere
npm install @langchain/community
```

```
npm install @langchain/google-vertexai
npm install  @langchain/ollama
```

LangChain 1.0 프레임워크 구성

langchain (메인 패키지)

- 에이전트 생성, 도구 정의, 일부 편의 기능 등 핵심 제공

```
import {
  createAgent,          // 에이전트 생성 (가장 중요한 팩토리)
  tool,                 // 도구 데코레이터 / 함수 (tool() 사용)
  initChatModel         // 통합 모델 초기화 (문자열로 여러 제공자 모델 생성 가능)
} from "langchain";
```

- 모델 초기화 initChatModel은 async 함수이므로 await 필수입니다 await initChatModel(...)
- tool은 zod 기반 스키마 사용

langchain-core

- 메시지, 프롬프트, Runnable 등 기본 추상화와 인터페이스를 제공

```
import {   SystemMessage,   HumanMessage,  AIMessage  } from "@langchain/core/messages";
import {   ChatPromptTemplate } from "@langchain/core/prompts";
```

LangChain 1.0 프레임워크 구성

프로바이더 패키지 (@langchain/openai, @langchain/anthropic등)

- 각 LLM 제공자와의 통합을 담당
- 각 패키지는 ChatXxx 클래스를 export하며, createAgent나 Runnable 체인에 직접 연결할 수 있습니다

```
npm install @langchain/openai
npm install @langchain/anthropic
npm install @langchain/google-genai
```

```
// OpenAI (gpt-4o, gpt-4o-mini 등)
import { ChatOpenAI } from "@langchain/openai";

// Anthropic (Claude 3.5, 3.7 Sonnet 등)
import { ChatAnthropic } from "@langchain/anthropic";

// Google Gemini 시리즈
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";

// Cohere (Command R 등)
import { ChatCohere } from "@langchain/cohere";

// 기타 예시
import { ChatGroq } from "@langchain/groq";            // Groq (Llama, Mixtral 등 고속 추론)
import { ChatFireworks } from "@langchain/fireworks"; // Fireworks AI
```

LangChain 1.0 프레임워크 구성

LangGraph

- 그래프(Graph) 기반으로 AI Agent 워크플로우를 설계하는 저수준 오케스트레이션 프레임워크
- 복잡한 에이전트 로직을 stateful로 유지하면서 제어 가능
- Stateful: 실행 중간에 상태가 자동으로 저장되고, 중단 후 재개 가능 (Checkpointing)
- Human-in-the-loop: 중간에 사람 개입 가능
- Streaming: 실시간으로 토큰/결과 스트리밍
- Persistence & Memory: 장기 메모리 지원
- Cyclic + Conditional: 단순 Chain으로는 불가능한 복잡한 로직(루프, 분기, 멀티에이전트) 구현 가능

State : 그래프 전체에서 공유되는 데이터 객체 (메시지 히스토리, 변수, 중간 결과 등)
Node : 실제 작업을 수행하는 함수. LLM 호출, Tool 실행, 일반 JS 로직 등 가능"할 일" 하나하나
Edge : 노드 간 연결과 흐름을 정의. 다음에 어떤 노드를 실행할지 결정
Graph : 노드 + 엣지로 구성된 전체 워크플로우

LangChain 1.0 프레임워크 구성

LangGraph

```
import { StateGraph, START, END } from "@langchain/langgraph";

// 1. State 정의
const AgentState = {
  messages: MessagesValue,   // 대화 기록
  nextAction: z.string().optional(),
};
// 2. 노드 정의 (함수)
const callModel = async (state) => { ... };
const useTool = async (state) => { ... };
const decideNext = (state) => { ... };   // conditional edge용
// 3. 그래프 구성
const graph = new StateGraph(AgentState)
  .addNode("agent", callModel)
  .addNode("tools", useTool)
  .addEdge(START, "agent")
  .addConditionalEdges("agent", decideNext, ["tools", END])  // 분기
  .addEdge("tools", "agent");   // 루프

const app = graph.compile();
```

```
npm install @langchain/langgraph
```

LangChain 1.0 프레임워크 구성

LangSmith

- Agent의 실행을 추적, 디버깅, 평가하는 관측성(Observability) 플랫폼
- LangChain.js와 완벽하게 통합되어 있으며, 자동으로 trace를 기록
- 커스텀 코드, 다른 프레임워크(OpenAI SDK 등)와도 연동 가능
- 자동 trace, playground에서 프롬프트 테스트, 평가(evals), 모니터링·알림 등 제공

LangSmith 활성화 방법 (환경 변수 설정만으로 대부분 완료) :

```
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY=lsv2_xxxxxxxxxxxx  # https://smith.langchain.com/ 에서 발급
export LANGCHAIN_PROJECT="my-js-agent-project"  # (옵션) 프로젝트 이름
```

```
import { LangChainTracer } from "langsmith";  // 또는 traceable wrapper
// invoke 시 callback 전달 가능
await agent.invoke(..., { callbacks: [new LangChainTracer()] });
```

LangChain 1.0 프레임워크 구성

기본 챗봇 : Runnable 체인 사용

1. LLM 모델 인스턴스 생성.
2. 프롬프트 템플릿 정의 (사용자 입력을 처리).
3. RunnableSequence로 체인 구성 : 프롬프트 → LLM → 출력 파서
4. 체인 호출로 응답 생성

```
npm install langchain @langchain/core @langchain/openai zod
npm install dotenv@^16.4.5
```

ESM 모드 사용 설정 (package.json)

```
{
  "type": "module"
}
```

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnableSequence } from "@langchain/core/runnables";
```

LangChain 1.0 프레임워크 구성

기본 챗봇: Runnable 체인 사용

- 프로젝트 루트에 .env 파일을 만들고 OpenAI API 키를 추가

```
// 1. LLM 모델 설정
const llm = new ChatOpenAI({ model: "gpt-4o-mini" });

// 2. 프롬프트 템플릿: 챗봇 역할을 정의
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful AI chatbot assistant."],
  ["human", "{input}"],  // 사용자 입력 플레이스홀더
]);

// 3. Runnable 체인 구성
const chain = RunnableSequence.from([
  prompt,                  // 입력 → 프롬프트
  llm,                     // 프롬프트 → LLM 호출
  new StringOutputParser(),  // LLM 출력 → 문자열 파싱
]);

// 4. 체인 호출 (챗봇 응답 생성)
const response = await chain.invoke({ input: "안녕? 오늘한국날씨 어때? 100자 아내로 응답해줘"});
console.log(response);  // 출력: 챗봇의 응답 문자열
```

```
C:₩langchain_dev>node simple-chatbot.js
```

LangChain 1.0 프레임워크 구성

고급 챗봇: createAgent 사용 (에이전트 기반)

1. tool 정의 : tool 함수로 함수와 스키마(zod) 정의.
2. createAgent로 에이전트 생성 : 모델과 툴 배열 전달.
3. Agent 호출 : 메시지 배열 입력으로 응답 생성 (tool 자동 호출).
4. (선택) 스트리밍이나 메모리 추가 : LangGraph로 확장.

```
import "dotenv/config";

import * as z from "zod";
import { createAgent, tool } from "langchain";
import { ChatOpenAI } from "@langchain/openai";

// 1. 툴 정의 (예: 날씨 조회 - 실제 API로 교체 가능)
const getWeather = tool(
  async ({ city }) => {
    // 실제 API 호출 로직
    return `${city}의  날씨는 화창합니다`;    //모의 데이터
  },
```

LangChain 1.0 프레임워크 구성

고급 챗봇: createAgent 사용 (에이전트 기반)

```
 {
    name: "get_weather",
    description: "Get the current weather for a city",
    schema: z.object({
      city: z.string().describe("The city name"),
    }),
  }
);

// 2. 모델 인스턴스 생성  ChatOpenAI 사용)
const model = new ChatOpenAI({
  model: "gpt-4o-mini",            // 또는 "gpt-4o", "gpt-4.1-mini" 등
  temperature: 0.7,
});
```

LangChain 1.0 프레임워크 구성

고급 챗봇: createAgent 사용 (에이전트 기반)

```
// 3. 에이전트 생성 → model 키로 전달
const agent = createAgent({
  model,
  tools: [getWeather],
  // (옵션) 추가 설정 가능
  // maxIterations: 10,
  // verbose: true,    // 디버깅용 로그 보기
});

// 4. 에이전트 호출 (메시지 배열 입력)
const response = await agent.invoke({
  messages: [
    { role: "system", content: "당신은 날씨예보 챗봇입니다. 필요하다면 tool을 사용합니다" },
    { role: "user", content: "오늘 서울 날씨를 100자이내로 요약해주세요" },
  ],
});

console.log(response.messages[response.messages.length - 1].content);  // 최종 응답 출력
```

```
C:₩langchain_dev>node agent-chatbot.js
```

LangChain Model

Messages

Messages

- 입력과 출력을 처리하는 기본적인 단위

메시지	역할	설명
SystemMessage	system	모델에게 전체 대화의 규칙, 페르소나, 응답 스타일, 제약 조건 등을 미리 알려줌 대화의 맥락, 역할, 제약사항 등을 설정 new SystemMessage("내용")
HumanMessage	user	사용자 입력, 질문, 대화 기록 중 인간의 발언 멀티모달 지원 시 이미지/파일 등도 포함 가능 new HumanMessage("내용")
AIMessage	assistant	LLM이 생성한 응답, 최종 답변, 중간 생각 과정 tool_calls 속성이 있으면 Agent가 Tool를 실행한 뒤 ToolMessage로 결과를 돌려줌 New AIMessage("내용")
ToolMessage	tool	도구(tool) 실행 결과가 LLM에게 돌아올 때 사용 tool_call_id가 반드시 일치해야 LLM이 어떤 도구 호출에 대한 결과인지 알 수 있음 에이전트(ReAct, OpenAI 함수 호출, LangGraph 등)에서 자동 생성됨 new ToolMessage({ content, tool_call_id, name })

Messages

Messages

메시지	역할	설명
ChatMessage	custom	커스텀 역할 메시지 (레거시/특수 용도) 특정 모델에서만 지원되는 역할 사용이 필요할 때 new ChatMessage("...", "custom_role")
AIMessageChunk	assistant	스트리밍 시 나오는 부분 응답 .stream() 또는 astream()에서 반환
ToolMessageChunk	tool	스트리밍 중 tool 결과의 부분 응답 , 거의 사용 안함

Messages

Messages

```
import { SystemMessage } from "@langchain/core/messages";

const system = new SystemMessage(
  "당신은 친절하고 정확한 한국어 수학 선생님입니다. " +
  "모든 답변은 단계별로 설명하고, 계산 과정도 보여주세요."
);
```

```
import { HumanMessage } from "@langchain/core/messages";

const human = new HumanMessage("2024년 한국 GDP 성장률은 몇 %였어?");
```

```
import { AIMessage } from "@langchain/core/messages";

const ai = new AIMessage({
  content: "2024년 한국 GDP 성장률은 약 2.2%였습니다.",
  // tool_calls가 있을 경우 예시
  // tool_calls: [{ name: "search_web", args: { query: "2024 한국 GDP" }, id: "call_abc123" }]
});
```

Messages

Messages

```
import { ToolMessage } from "@langchain/core/messages";

const toolResult = new ToolMessage({
  content: "검색 결과: 2024년 한국 GDP는 약 1,870조 원 (명목 기준)",
  tool_call_id: "call_abc123",    // AIMessage의 tool_calls[].id 와 일치해야 함
  name: "search_web"              // (선택) 어떤 도구인지 표시
});
```

Messages

Messages

- Message의 구성 순서 : SystemMessage → Few-shot 예시 → chat_history(대화이력) → 현재 HumanMessage (사용자 최신 질문)

```
import {  SystemMessage,  HumanMessage,  AIMessage,  ToolMessage ,} from "@langchain/core/messages";
import {  ChatPromptTemplate,  MessagesPlaceholder, } from "@langchain/core/messages";

const messages = [
  new SystemMessage(`너는 세계 최고의 한국어 선생님이야.
항상 존댓말을 사용하고, 3문장 이내로 답변해.`),

  // Few-shot (선택)
  new HumanMessage("자음 탈락이 뭐야?"),
  new AIMessage("말할 때 'ㄱ'이나 'ㅂ' 같은 자음이 탈락하는 현상이에요. 예: '먹고' → '멱고'"),

  // 대화 이력 (메모리에서 불러옴)
  new MessagesPlaceholder("chat_history"),

  // 현재 질문
  new HumanMessage("{question}"),
];
```

Messages

HumanMessages 실습 코드

```
import "dotenv/config";
import fs from "fs";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage } from "@langchain/core/messages";

async function main() {
  // 1. 이미지 Base64 인코딩
  const imageBuffer = fs.readFileSync("./비숑.jpg");
  const base64Image = imageBuffer.toString("base64");

  // 2. 모델 생성
  const model = new ChatOpenAI({
    model: "gpt-4o",
    temperature: 0,
  });
```

Messages

HumanMessages 실습 코드

```
  // 3. 멀티모달 메시지 구성
 const message = new HumanMessage({
  content: [
    { type: "text", text: "이 이미지에 무엇이 있나요?" },
    {
      type: "image_url",
      image_url: {
        url: `${base64Image}`,
      },
    },
  ],
});

const response = await model.invoke([message]);

console.log(response.content);
}

main();
```

C:₩langchain_dev>npx tsx humanmessage_example.ts

Messages

AIMessages

- AIMessage는 도구 호출 정보를 포함할 수 있습니다
- 모델에 메시지 리스트를 전달하여 대화 컨텍스트를 구성할 수 있습니다
- 메시지 객체 대신 딕셔너리 형식을 사용하여 대화 컨텍스트를 구성할 수 있습니다

C:₩langchain_dev>npx tsx aimessage-example.ts

AIMessage + tool_calls 실습

C:₩langchain_dev>npx tsx toolmessage-example.ts

AIMessage → 도구 실행 → ToolMessage → 다시 모델 호출 실습

LangChain Model

LLM

- 사용자가 특정 주제에 대한 설명을 요청할 때, LLM은 주어진 텍스트 입력을 바탕으로 상세한 설명을 생성하여 반환
- 단일 문자열 입력 → 단일 문자열 출력
- 광범위한 언어 이해 및 텍스트 생성 작업에 사용
- 문서 요약, 콘텐츠 생성, 질문에 대한 답변 생성 등 복잡한 자연어 처리 작업을 수행
- Completion 스타일
- LLM 계열은 completion 모델(text-davinci-003, gpt-3.5-turbo-instruct, babbage-002)이 deprecated되면서 거의 사용되지 않습니다.
- LLM(GPT-4o, GPT-4o-mini, Claude 3/4 시리즈, Gemini 1.5/2.0, Llama 3.1/4 등)은 Chat Completion 모델

Completion 스타일 = "다음 단어(토큰) 예측" 방식으로 텍스트를 생성하는 전통적인 LLM 사용 방식
모델에게 하나의 긴 프롬프트(prompt)만 던져주고, 모델이 그 뒤를 이어서 계속 완성(completion)해 나가는 스타일
대화 형식(messages)이 아니라 단일 문자열 prompt만 사용
대화 히스토리를 유지하려면 사용자가 직접 이전 대화를 모두 prompt에 붙여넣어야 함
시스템 지시, 사용자 메시지, 어시스턴트 메시지를 구분하기 어려움

LangChain Model

LLM Chat Model

- 대화형 상황에 최적화되어 있으며, 사용자와의 연속적인 대화를 처리하는 데 사용
- 메시지의 리스트를 입력으로 받고, 하나의 메시지를 반환
- BaseLanguageModel (@langchain/core/language_models/base) → LLM과 ChatModel의 공통 부모 (추상 클래스)
- BaseLLM (@langchain/core/llms) → 추상클래스
- Message[] (Human, System, AI 등 메시지 배열) 입력 → AIMessage / ChatMessage대화형 출력
- BaseChatModel, ChatModel (@langchain/core/chat_models)

LangChain Model

LLM ChatModel 제공자 패키지

클래스	패키지	환경변수	예시
ChatOpenAI	@langchain/openai	OPENAI_API_KEY	GPT-4o, o1-mini, GPT-4o-mini, o1-preview
ChatAnthropic	@langchain/anthropic	ANTHROPIC_API_KEY	Claude 3.5 Sonnet, Claude 3 Opus, Haiku
ChatGoogleGenerativeAIG	@langchain/google-genai	GOOGLE_API_KEY 또는 GEMINI_API_KEY	google_genai:gemini-2.5-flash-lite
ChatCohere	@langchain/cohere	COHERE_API_KEY	Gemini 1.5 Pro, Gemini 1.5 Flash
ChatGroq	@langchain/groq	GROQ_API_KEY	Llama 3.1 405B, Mixtral, Gemma2 등
ChatOllama	@langchain/community (Ollama)	없음(로컬)	Llama 3.1, Phi-3, Gemma2 로컬 실행
ChatCloudflareWorkersAI	@langchain/cloudflare	CLOUDFLARE_API_TOKEN / CLOUDFLARE_ACCOUNT_ID	Llama 3, Mistral, Gemma 등 edge
ChatBedrock, BedrockLLM	@langchain/aws (Bedrock)		Claude 3, Llama 3, Titan, Cohere 등

LangChain Model

LLM ChatModel 공통 특성

- 호출 방식 (LCEL 스타일)

```
const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });
const response = await model.invoke("안녕하세요!");
// 또는 chain: prompt.pipe(model).pipe(parser)
```

- 스트리밍

```
for await (const chunk of model.stream(input)) {
  process.stdout.write(chunk.content as string);
}
```

- Tool / Function Calling

```
const modelWithTools = model.bindTools(tools);
```

- Structured Output (Zod / JSON schema 강제)

```
const structured = model.withStructuredOutput(z.object({...}));
```

LangChain Model

LLM ChatModel 공통 특성

- System / User 메시지 간단 표기

```
await model.invoke([
  ["system", "너는 친절한 한국어 선생님이다."],
  ["human", "자음 탈락에 대해 설명해줘"],
]);
```

- Configurable fields (temperature, maxTokens, topP 등 런타임 변경 가능)

```
const chain = prompt.pipe( model.withConfig({ temperature: 0.2 }) );
```

LangChain Model

LLM ChatModel 공통 파라미터

파라미터	패키지
model / modelName	사용할 모델 이름
temperature	창의성/무작위성 정도. 0에 가까울수록 deterministic, 높을수록 창의적 0.0 (정확), 0.7 (균형), 1.2 (창의)
maxTokens/max_tokens	한 번에 생성할 최대 토큰 수
topP / top_p	nucleus sampling. 누적 확률 topP% 내에서만 샘플링 temperature와 함께 쓰지 말라는 권장 많음
topK	top-k sampling (주로 Groq, 일부 모델만)
frequencyPenalty	값이 클수록 반복 단어나 구절을 줄이고 텍스트의 다양성을 증가시킴 (0~1)
presencePenalty	값이 클수록 텍스트에 등장하지 않은 새로운 단어를 사용, 선택 확률을 조정 (0~1)
seed	재현성 위한 시드 값 (일부 모델만)
streaming	스트리밍 모드 활성화 여부
maxRetries	API 호출 실패 시 재시도 횟수

LangChain Model

LLM ChatModel 공통 파라미터

파라미터	패키지
timeout	생성할 최대 토큰 수를 지정합니다. 생성할 텍스트의 길이를 제한합니다.
apiKey / 제공자별 키	API 키 (환경 변수 우선

LangChain Model

ChatModel 공통 파라미터

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-4o-mini",                // 또는 modelName
  temperature: 0.3,                   // 낮을수록 사실적/보수적
  maxTokens: 2048,
  topP: 0.95,
  frequencyPenalty: 0.2,               // 반복 줄이기
  presencePenalty: 0.1,
  maxRetries: 3,
  timeout: 60000,                     // 60초
  streaming: true,                    // .stream() 사용 시 true 추천
  // openAIApiKey: "...",             // 직접 넣을 때 (비추천)
});
```

LangChain Model

ChatModel 공통 파라미터

```
import { ChatAnthropic } from "@langchain/anthropic";

const claude = new ChatAnthropic({
  model: "claude-3-5-sonnet-20241022",
  temperature: 0.5,
  maxTokens: 4096,                    // Claude는 max_tokens 거의 필수
  anthropicApiKey: process.env.ANTHROPIC_API_KEY,
});
```

```
import { ChatGroq } from "@langchain/groq";

const groq = new ChatGroq({
  model: "llama-3.1-70b-versatile",
  temperature: 0.0,                   // Groq는 0으로도 빠름
  maxTokens: 8192,
  topP: 0.9,
});
```

LangChain Model

BaseLanguageModel / BaseChatModel 공통 파라미터

파라미터	패키지
cache	boolean | BaseCache → 응답 캐싱 여부 (기본 false)
callbacks	Callbacks → LangSmith/LangFuse 등 추적용
tags	string[] → 추적 태그
metadata	Record<string, unknown> → 추가 메타데이터
verbose	boolean → 디버그 로그 상세도 (기본 false)

LangChain Model

런타임에서 파라미터 동적으로 변경

- withConfig() : 모델의 실행 옵션(런타임 설정)을 추가하거나 재정의한 새로운 Runnable 인스턴스를 반환하는 함수

```
// chain 수준에서 temperature만 바꾸기
const chain = prompt.pipe(model.withConfig({ temperature: 0.1 }));
```

- bind() : 모델 인스턴스에 파라미터를 추가로 제공
  특정 모델 설정을 기본값으로 사용하고자 할 때 유용하며, 특수한 상황에서 일부 파라미터를 다르게 적용하고 싶을 때 사용

```
//  bind로 tool + 파라미터 고정
const modelWithTools = model.bind({
  tools: [...],
  temperature: 0.2,
  maxTokens: 512,
});
```

LangChain Model

Model 실행 메서드

메서드	반환타입	용도
invoke(input, config?)	Promise<BaseMessage> (AIMessage)	단일 입력 → 전체 응답 한 번에 받기
stream(input, config?)	AsyncIterable<BaseMessageChunk>	실시간 스트리밍 (chunk 단위로 조금씩 받음)
batch(inputs, config?)	Promise	여러 입력을 동시에 처리 (병렬 호출)
abatch(inputs, config?)	Promise<BaseMessage[]>	batch의 async 버전 (거의 batch와 동일)
generate(messagesList)	Promise<LLMResult>	내부적으로 batch처럼 여러 프롬프트를 한 번에 처리 (invoke/batch로 대체) 커스텀 모델 구현 시 또는 배치 최적화
astreamEvents(...)	AsyncIterable<StreamEvent>	매우 상세한 스트리밍 (이벤트 단위: on_chain_start, on_llm_new_token 등)

LangChain Model

Model 실행 메서드

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({ model: "gpt-4o-mini" });

// 1. invoke() - 전체 한 번에
const fullResponse = await model.invoke("안녕! 한국의 4대 계절을 설명해줘.");
console.log(fullResponse.content);  // 한 번에 다 나옴

// 2. stream() - 실시간 chunk
const stream = model.stream("한국의 4대 계절을 하나씩 천천히 설명해줘.");
for await (const chunk of stream) {
  process.stdout.write(chunk.content?.toString() ?? "");  // 한 글자씩 출력됨
}

// 3. batch() - 여러 질문 동시에
const results = await model.batch([
  "서울 날씨 어때?",
  "부산 날씨 어때?",
]);
console.log(results.map(r => r.content));
```

LangChain Model

Streaming

- LLM의 응답을 토큰 단위로 실시간 수신하는 기능
- 긴 응답을 기다리지 않고 즉시 사용자에게 피드백을 제공할 수 있어 UX 향상
- AI의 응답을 작은 Chunk단위(AIMessageChunk 객체)로 실시간 생성하고 반환
- LCEL 체인에서도 동일하게 스트리밍을 사용할 수 있습니다

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage } from "@langchain/core/messages";

async function streamPoem() {
  const model = new ChatOpenAI({
    model: "gpt-4o-mini",
    temperature: 0.9,
    maxTokens: 150,
    streaming: true
  });
```

LangChain Model

Streaming

```
async function generatePoem(theme: string) {
    const stream = await model.stream([
      new HumanMessage(`주제 '${theme}'로 짧은 시를 써주세요.`)
    ]);

    for await (const chunk of stream) {
      if (typeof chunk.content === "string") {
        process.stdout.write(chunk.content);
      }
    }
    process.stdout.write("₩n");
  }

  await generatePoem("가을 낙엽");
  await generatePoem("첫사랑의 기억");
}

//  함수 호출 추가
streamPoem().catch(console.error);
```

```
C:₩langchain_dev>npx tsx stream-poem.ts
```

LangChain Model

streaming 옵션

- 런타임에 동적으로 변경 (withConfig) 스트리밍을 활성화/비활성화

```
// chain 수준에서 streaming 끄기/켜기
const chain = prompt.pipe(model);

const withStreaming = chain.withConfig({ streaming: true });
const withoutStreaming = chain.withConfig({ streaming: false });
```

```
// 또는 bind로 고정
const boundModel = model.bind({ streaming: true });
```

- timeout은 HTTP 요청 타임아웃으로 작동

```
const model = new ChatOpenAI({
  model: "gpt-4o-mini",
  streaming: true,          // ← streaming 활성화
  timeout: 60000,            // ← 60초 timeout (밀리초 단위)
  temperature: 0.7,
  // maxRetries: 2,         // 필요 시 추가
});
```

LangChain Model

통합 모델 초기화

- LangChain은 각 프로바이더마다 별도의 클래스(ChatOpenAI, ChatAnthropic 등)를 사용할 수 있지만, initChatModel 함수를 사용하면 통합된 인터페이스로 모든 프로바이더의 모델을 초기화할 수 있습니다.
- initChatModel : 모델 제공자를 추상화한 팩토리 함수
- 다양한 LLM 제공자를 하나의 통일된 인터페이스로 다룰 수 있게 해줍니다.
- 모델 제공자를 자주 바꾸거나, 설정 파일로 모델을 동적으로 선택해야 할 때 특히 유용합니다.
- 지원 provider → openai, anthropic, groq, google_vertexai, cohere, mistralai, ollama 등

```
import { initChatModel } from "langchain";   // 또는 "@langchain/langchain" 패키지

const model = await initChatModel("gpt-4o-mini", {
  temperature: 0.7,
  maxTokens: 4096,
  // provider를 명시하지 않으면 openai로 추정 (환경변수 기반)
});

const claude = await initChatModel("claude-3-5-sonnet-20241022", {
  provider: "anthropic",
  temperature: 0.5,
  maxTokens: 8192,
});
```

LangChain Model

initChatModel 파라미터

파라미터	설명
첫번째 인자(model)	사용할 모델 이름 또는 식별자
provider	LLM 제공자 이름
temperature	창의성/무작위성
maxTokens / max_tokens	한 번에 생성할 최대 토큰 수
topP / top_p	nucleus sampling (누적 확률 기준 샘플링)
frequencyPenalty	같은 토큰 반복 확률
presencePenalty	새로운 토큰 등장 확률
streaming	스트리밍 모드 기본 활성화 여부
timeout	요청 전체 타임아웃 (밀리초)
maxRetries	API 호출 실패 시 재시도 횟수
baseUrl / apiBase	커스텀 API 엔드포인트 (Ollama, vLLM, 로컬 서버 등)
apiKey	API 키 (환경변수보다 우선 적용)

LangChain Model

initChatModel 파라미터

파라미터	설명
configuration	제공자별 추가 설정 (OpenAI SDK의 configuration 등
cache	응답 캐싱 여부
callbacks	LangSmith, LangFuse 등 추적용 콜백
verbose	디버그 로그 상세 출력

Prompt & Chain

Prompt

PromptTemplate

- 사용자의 입력과 다양한 매개변수를 대규모 언어 모델(LLM)이 이해할 수 있는 지시사항으로 변환하는 도구
- 단순 질의, 요약, 번역, 설명 생성을 위해 사용
- "지시", "예시", "맥락", "질문" 과 같은 다양한 구성요소들을 조합할 수 있습니다

구분	내용 & 예시
지시	언어 모델에게 어떤 작업을 수행하도록 요청하는 구체적인 지시 "아래 제공된 제품 리뷰를 요약해주세요."
예시	요청된 작업을 수행하는 방법에 대한 하나 이상의 예시 "예를 들어, '이 제품은 매우 사용하기 편리하며 배터리 수명이 길다'라는 리뷰는 '사용 편리성과 긴 배터리 수명이 특징'으로 요약할 수 있습니다."
맥락	특정 작업을 수행하기 위한 추가적인 맥락 "리뷰는 스마트워치에 대한 것이며, 사용자 경험에 초점을 맞추고 있습니다."
질문	어떤 답변을 요구하는 구체적인 질문 "이 리뷰를 바탕으로 스마트워치의 주요 장점을 두세 문장으로 요약해주세요."

Prompt

Prompt 작성 원칙

원칙	설명
명확성	모호함 제거, 구체적 지시, 상세한 템플릿 작성 예시] "설명해줘" → "3가지 핵심 포인트로 설명해줘"
구체성	세부 사항과 제약 조건 명시 예시] "요약해줘" → "200자 이내로 요약해줘"
컨텍스트	필요한 배경 정보 제공, System 메시지 활용 예시] 대상 청중, 사용 목적 명시
구조화	역할, 지시, 예시 분리, ChatPromptTemplate 예시] System/Human 메시지 분리
출력 형식	원하는 응답 형식 지정 예시] JSON, 표, 번호 목록 등
예시 제공	Few-shot으로 패턴 학습, FewShotChatMessagePromptTemplate 예시] 입력-출력 예시 포함
환각 방지	불확실성 인정 유도, 제약조건 명시

Prompt

PromptTemplate

- 단일 문장 또는 간단한 명령을 입력하여 단일 문장 또는 간단한 응답을 생성하는 데 사용되는 프롬프트
- 변수를 포함한 문자열 템플릿으로, 동적으로 프롬프트를 생성할 수 있게 함
- fromTemplate()를 사용하여 문자열 템플릿으로부터 PromptTemplate 인스턴스를 생성
- format()를 사용하여, 변수에 값으로 템플릿에 채워서 프롬프트를 구성

```
import { PromptTemplate } from "@langchain/core/prompts";

// 템플릿 정의
const template = PromptTemplate.fromTemplate(
  "{topic}에 대해 {length}자 이내로 설명해주세요."
);

// 변수 대입
const prompt = await template.format({
  topic: "인공지능",
  length: "100",
});

console.log(prompt);
```

Prompt

PromptTemplate

- + 연산자를 사용하여 직접 결합하는 동작을 지원
- PromptTemplate 인스턴스 간의 직접적인 결합, 인스턴스와 문자열로 이루어진 템플릿을 결합하여 새로운 PromptTemplate 인스턴스를 생성하는 것도 가능

```
// 기본 템플릿 정의
const introPrompt = new PromptTemplate({
  template: "당신은 {role} 입니다.₩n",
  inputVariables: ["role"],
});
const questionPrompt = new PromptTemplate({
  template: "다음 질문에 답해주세요:₩n{question}",
  inputVariables: ["question"],
});
// PromptTemplate 간 직접 결합 (문자열 추출 후 +)
const combinedTemplateString =  introPrompt.template + questionPrompt.template;
// 새로운 PromptTemplate 생성
const combinedPrompt = new PromptTemplate({
  template: combinedTemplateString,
  inputVariables: ["role", "question"],
});
```

```
C:₩langchain_dev>npx tsx prompt_example.ts
```

Prompt

ChatPromptTemplate

- 대화형 상황에서 여러 메시지 입력을 기반으로 단일 메시지 응답을 생성하는 데 사용
- 복잡한 구조 + 유연성 + 간편한 포맷
- 챗봇 개발에 주로 사용
- role 기반 메시지 구성 (system / human / ai)
- 역할 기반 지시, 전문 페르소나 설정, 멀티턴 구성 가능

```
// Prompt 정의
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "너는 세계 최고의 {job} 전문가야."],
  ["human", "다음 주제에 대해 알려줘: {topic}"],
  new MessagesPlaceholder("conversation"),
]);

// 모델 생성
const model = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0.3,
  maxTokens: 200,
});
```

```
C:₩langchain_dev>npx tsx chatprompt_example.ts
```

Prompt

- **ChatPromptTemplate**
  - 제약 조건 명시

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";

const prompt = ChatPromptTemplate.fromMessages([
[
"system",
`당신은 기술 문서 작성 전문가입니다.

응답 규칙:
- 문장은 간결하게 (20단어 이내)
- 능동태 사용
- 전문 용어 사용 시 괄호 안에 설명 추가
- 총 200자 이내로 작성`,
],
["human", "{question}"],
]);
const model = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0.2,
  maxTokens: 150,   // 출력 길이 추가 제한
});
const chain = prompt.pipe(model);
```

Prompt

ChatPromptTemplate

- 컨텍스트 제공 - 대상 청중 명시

```
 const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0.4,
  maxTokens: 300,
});

const prompt =
ChatPromptTemplate.fromMessages([
  [
    "system",
    `당신은 교육 콘텐츠 전문가입니다.

대상 청중: {audience}
- 초보자: 비유와 일상 예시 사용, 전문 용어 최소화
- 중급자: 개념과 실습 코드 병행
- 전문가: 심화 내용과 최적화 기법 포함`,
  ],
  ["human", "{topic}에 대해 설명해주세요."],
]);
```

```
const chain = prompt.pipe(llm);
 const beginner = await chain.invoke({
    audience: "초보자",
    topic: "API",
  });

  const expert = await chain.invoke({
    audience: "전문가",
    topic: "API",
  });
```

Prompt

ChatPromptTemplate

- 구조화된 프롬프트 - 단계별 지시사항

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "당신은 코드 리뷰 전문가입니다."],
  [
    "human",
    `다음 코드를 리뷰해주세요:

<code>
{code}
</code>

다음 순서로 분석해주세요:
1. **기능 요약**: 코드가 수행하는 작업 설명
2. **장점**: 잘 작성된 부분 (2-3가지)
3. **개선점**: 수정이 필요한 부분 (2-3가지)
4. **리팩토링 제안**: 개선된 코드 예시

각 섹션은 명확히 구분하여 작성해주세요.`,
  ],
]);
```

Prompt

MessagesPlaceholder

- 템플릿 내에서 유동적으로 메시지를 삽입할 수 있는 자리를 미리 지정해 두는 데 사용
- 챗봇 시스템에서 사용자와 AI 사이의 대화(Chat History)를 ChatPromptTemplate에 동적으로 삽입하는데 활용
- [시스템 메시지, [대화 기록], 사용자 입력] 형태로 정의
- Conversational Agent 필수 구성요소
- 메모리 연동 가능
- 챗봇, FAQ Bot, Agent 구현에 사용

```
import { MessagesPlaceholder } from "@langchain/core/prompts";

const prompt = ChatPromptTemplate.fromMessages([
  ["system", "당신은 친절한 튜터입니다."],
  new MessagesPlaceholder("history"),
  ["human", "{question}"]
]);
```

Prompt

Few-shot Template

- Few-shot 학습은 언어 모델에 몇 가지 예시를 제공하여 특정 작업을 수행하도록 유도하는 기법
- Few-shot 학습을 활용함으로써, 언어 모델은 주어진 예제들을 참고하여 더 정확하고 일관된 응답을 생성할 수 있습니다.
- 특히 특정 도메인이나 형식의 질문에 대해 모델의 성능을 향상시키는 데 효과적
- FewShotPromptTemplate 은 예제들을 결합하고 새로운 입력을 추가하여 최종 프롬프트를 생성합니다.

```
import { PromptTemplate, FewShotPromptTemplate } from "@langchain/core/prompts";

const examplePrompt = PromptTemplate.fromTemplate(
  "질문: {question}₩n{answer}"
);
const examples = [
{
question: "지구의 대기 중 가장 많은 비율을 차지하는 기체는 무엇인가요?",
answer: "지구 대기의 약 78%를 차지하는 질소입니다.",
},
{
question: "광합성에 필요한 주요 요소들은 무엇인가요?",
answer: "광합성에 필요한 주요 요소는 빛, 이산화탄소, 물입니다.",
},
```

```
C:₩langchain_dev>npx tsx fewshotprompt_example.ts
```

Prompt

PipelinePromptTemplate

- 여러 프롬프트를 연결
- 프롬프트 단계별 구성
- 복잡한 체인 구성
- LCEL 기반 파이프라인 설계에 적합
- 요약 → 재작성 → 번역 , 단계적 처리에 사용

Prompt

Partial Prompt

- 필요한 값의 일부만 미리 입력하여 새로운 프롬프트 템플릿
- 일부 변수를 미리 바인딩
- 공통 변수 고정
- 재사용성 향상
- Agent 설계 시 유용

```
const basePrompt = PromptTemplate.fromTemplate(
  "당신은 {role}입니다. 질문: {question}"
);

const partial = await basePrompt.partial({
  role: "MySQL 전문가"
});

const result = await partial.invoke({
  question: "Redo Log란?"
});
```

Prompt

PromptTemplate

- 환각 방지

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.fromMessages([
    ["system", '당신은 정확한 정보만 제공하는 AI입니다.

중요한 규칙:
1. 확실하지 않은 정보는 "확실하지 않습니다"라고 말하세요
2. 추측이 필요한 경우 "추측입니다만..."으로 시작하세요
3. 최신 정보가 필요한 질문은 검증이 필요함을 안내하세요
4. 출처가 있다면 명시하세요'),
    ["human", "{question}"]
])
```

Prompt

Partial Prompt

- 함수를 사용한 부분 포맷팅 - 동적으로 값을 생성.

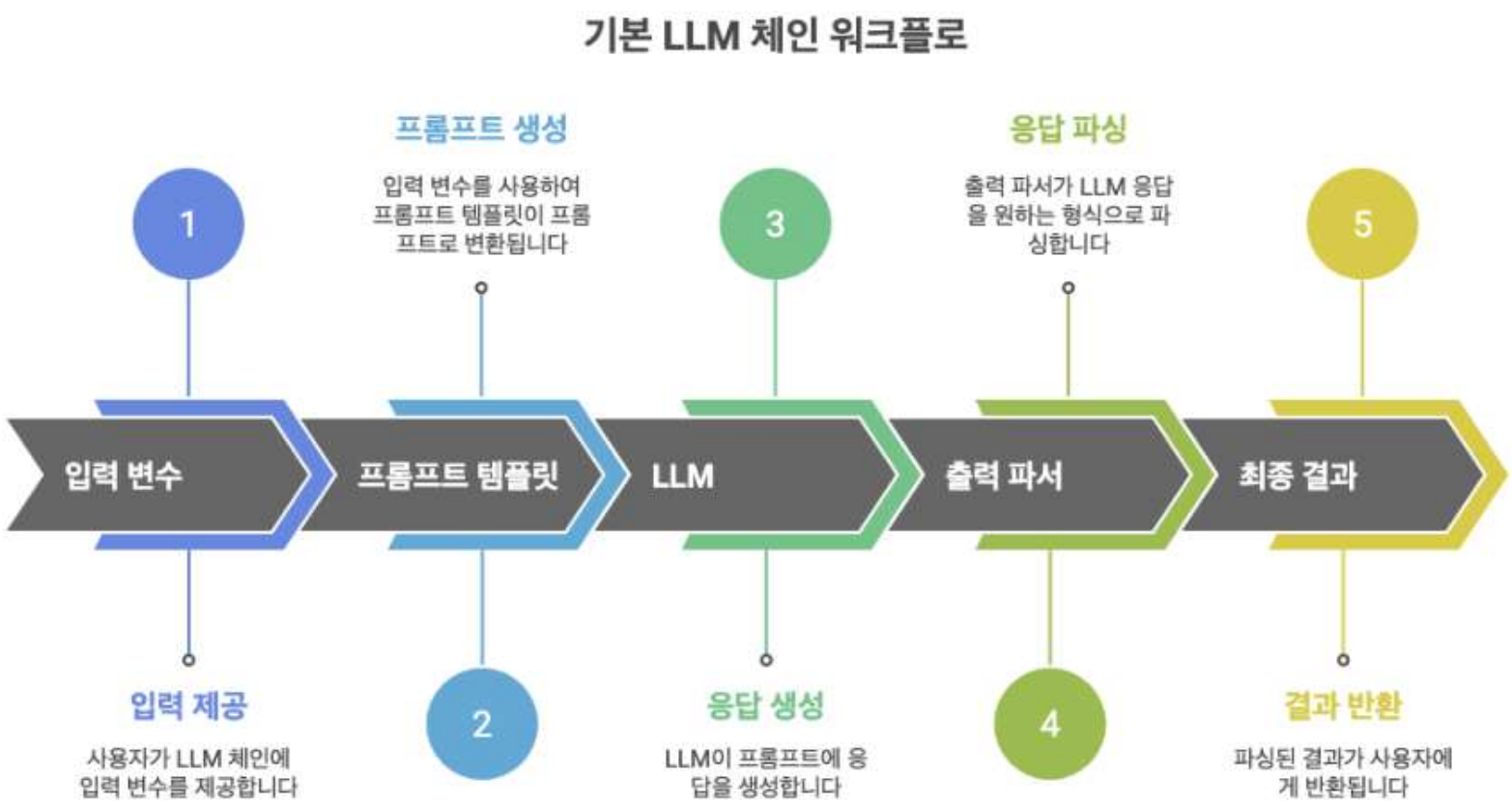
```
//현재 계절 구하는 함수
function getCurrentSeason() {
  const month = new Date().getMonth() + 1; // 0~11 이므로 +1

  if (month >= 3 && month <= 5) return "봄";
  else if (month >= 6 && month <= 8) return "여름";
  else if (month >= 9 && month <= 11) return "가을";
  else return "겨울";
}
async function run() {
  // PromptTemplate 생성
  const prompt = PromptTemplate.fromTemplate(
    "{season}에 일어나는 대표적인 지구과학 현상은 {phenomenon}입니다."
  );
  // partial 적용 (동적 함수 실행)
  const partialPrompt = await prompt.partial({
    season: getCurrentSeason(), // 함수 실행 결과를 바인딩
  });
```

C:₩langchain_dev>npx tsx partialprompt.ts

Chain

Chain

- 여러 컴포넌트를 연결하여 하나의 워크플로우를 구성하는 것



Chain

Runnable

- LCEL(LangChain Expression Language)의 가장 핵심이 되는 추상화 인터페이스/프로토콜
- Prompt, Model, Output Parser, Retriever, Tool, Chain 모두 Runnable 프로토콜을 구현
- LangGraph의 Node도 내부적으로 Runnable을 기반으로 동작합니다.
- 여러 Runnable을 연결해 체인(RunnableSequence)을 만들 수 있으며, Prompt → LLM 호출 → Output Parser의 흐름을 정의합니다.

```
RunnableSequence.from([prompt, llm, parser])
```

```
Runnable 프로토콜을 구현 객체:
ChatOpenAI / ChatAnthropic / ChatGoogleGenerativeAI 등 → LLM
ChatPromptTemplate
StringOutputParser / JsonOutputParser / StructuredOutputParser
RunnableSequence
RunnableParallel
RunnableBranch
RunnableLambda / RunnablePassthrough
Retriever (vectorStore.asRetriever())
Tool / StructuredTool
createReactAgent() 결과물 등
```

Chain

Runnable

- LCEL로 구성된 Runnable (체인, 모델, 파서, 리트리버 등 대부분의 구성 요소)들은 실행 메서드를 표준으로 제공

메서드	비동기 버전	설명
invoke()	ainvoke()	단일 입력에 대해 체인을 호출하고 결과를 반환
batch()	abatch()	여러 입력을 한 번에 처리 (병렬 가능) , 결과를 배열로 반환, 대량 처리, 배치 처리
stream()	astream()	토큰 단위로 실시간 스트리밍 출력 (청크 단위로 yield), AsyncIterable (청크 스트림) 반환 UI 실시간 표시, 긴 응답
	astreamEvents()	이벤트 단위 스트리밍 (on_chain_start, on_llm_new_token 등 상세 이벤트) AsyncIterable<Event> 반환 디버깅·로깅·복잡한 UI
	astreamLog()	LangServe 호환 로그 스트리밍 (중간 단계 포함) AsyncIterable<RunLogPatch> 반환 LangServe / LangSmith 연동

Chain

LCEL (LangChain Expression Language)

- 체인을 파이프라인처럼 연결할 수 있다
- 체인의 모든 구성 요소가 Runnable 인터페이스를 구현
- .pipe(), .withConfig(), .withFallbacks(), 병렬 처리(RunnableParallel), 조건 분기(RunnableBranch) 등을 체인처럼 조합 가능
- 스트리밍, 비동기, 배치 처리가 기본 지원

LCEL 구성 요소	용도
.pipe()	순차 연결 prompt.pipe(model).pipe(parser)
RunnableSequence.from()	여러 runnable을 배열로 연결 RunnableSequence.from([prompt, model, parser])
RunnablePassthrough	입력 그대로 전달 (키 매핑에 자주 사용) { question: new RunnablePassthrough(), context }
RunnableParallel	병렬 실행 (여러 체인 동시에) RunnableParallel.from({ summary, sentiment })
RunnableBranch	조건 분기 RunnableBranch.from([[condition, chainA], chainB])

Chain

LCEL (LangChain Expression Language)

LCEL 구성 요소	용도
.withConfig()	태그, 메타데이터, 최대 재시도 등 설정 chain.withConfig({ tags: ["v1", "korean"] })
.withFallbacks()	대체 체인 (에러 시) chain.withFallbacks([backupChain])
.batch() / .stream()	배치 / 스트리밍 실행 await chain.stream(input)

Chain

LCEL (LangChain Expression Language)

```
import "dotenv/config";

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnableSequence, RunnablePassthrough } from "@langchain/core/runnables";

// 1. 기본적인 LCEL 체인
const model = new ChatOpenAI({ modelName: "gpt-4o-mini", temperature: 0 });

const prompt = ChatPromptTemplate.fromMessages([
  ["system", "당신은 훌륭한 한국어 번역가입니다."],
  ["human", "{text}를 한국어로 자연스럽게 번역해 주세요."],
]);

const chain = prompt.pipe(model).pipe(new StringOutputParser());

const result = await chain.invoke({ text: "The quick brown fox jumps over the lazy dog." });
console.log(result);
```

Chain

Chain 배치 처리

- Chain (여러 컴포넌트를 연결한 결과) : 체이닝 자체가 Runnable을

C:₩langchain_dev>npx tsx chain_batch_example.ts

```
import "dotenv/config";

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

const model = new ChatOpenAI({ model: "gpt-4o-mini" });
const prompt = ChatPromptTemplate.fromTemplate("한국어로 번역해줘: {text}");
const chain = prompt.pipe(model).pipe(new StringOutputParser());

// batch / abatch
const results = await chain.batch([
  { text: "Yesterday I was clever, so I wanted to change the world. Today I am wise, so I am changing
myself." },
  { text: "In the end, it's not the years in your life that count. It's the life in your years." },
]);
console.log(results);
```

Chain

Chain stream 처리

```
import "dotenv/config";

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

// stream / astream
async function main() {
  const model = new ChatOpenAI({
    model: "gpt-4o-mini",
  });
  const prompt = ChatPromptTemplate.fromTemplate(
    "한국어로 번역해줘: {text}"
  );
  const chain = prompt.pipe(model);
  const stream = await chain.stream({
    text: "Your time is limited, so don't waste it living someone else's life.",
  });
  for await (const chunk of stream) {
    process.stdout.write(chunk.content);
  }
}

main();
```

```
C:₩langchain_dev>npx tsx chain_stream_example.ts
```

Chain

ChatPromptTemplate.fromTemplate() 또는 fromMessages()

- 문자열 형태의 템플릿을 인자로 받아, 해당 형식에 맞는 프롬프트 포맷팅을 생성

```
const prompt = ChatPromptTemplate.fromTemplate(
  "당신은 {role}입니다. 다음 주제에 대해 설명해 주세요: {topic}"
);

const prompt2 = ChatPromptTemplate.fromMessages([
  ["system", "당신은 전문 {job}입니다."],
  ["human", "{question}"],
]);
```

Chain

Sequential Chain

- 단계별 순차 처리

```
const chain = classificationPrompt
  .pipe(classificationModel)
  .pipe(new StringOutputParser())
  .pipe(routingMap)                    // router
  .pipe(RunnableBranch.from(...))      // 분기
  .pipe(finalModel)
  .pipe(new StringOutputParser());
```

Chain

다단계 Sequential Chain

- RunnableSequence 또는 .pipe() 여러 번 연결
- 4~10단계 이상의 복잡한 순차 로직 : RunnableSequence.from([a, b, c, d, ...])

```
const longChain = RunnableSequence.from([
  step1,          // 입력 → 전처리
  step2,          // 검색
  step3,          // 문서 압축
  step4,          // 프롬프트
  step5,          // LLM
  step6,          // 출력 파싱
  step7,          // 후처리 (검증 등)
]);
```

```
 const chain = step1
  .pipe(step2)
  .pipe(step3)
  .pipe(step4)
  .pipe(step5)
  .pipe(step6)
  .pipe(step7);
```

Chain

Parallel Chain (RunnableParallel)

- RunnableParallel : 여러 Runnable을 동시에(병렬로) 실행할 수 있게 해주는 클래스
- RunnableParallel 은 객체 형태로 여러 키를 주면, 각 키마다 독립적으로 Runnable을 실행하고, 결과를 객체로 모아서 반환합니다.
- 여러 작업을 동시에 수행(병렬 처리)하고 결과를 딕셔너리로 모음 : RunnableParallel.from({ key1: chainA, key2: chainB })

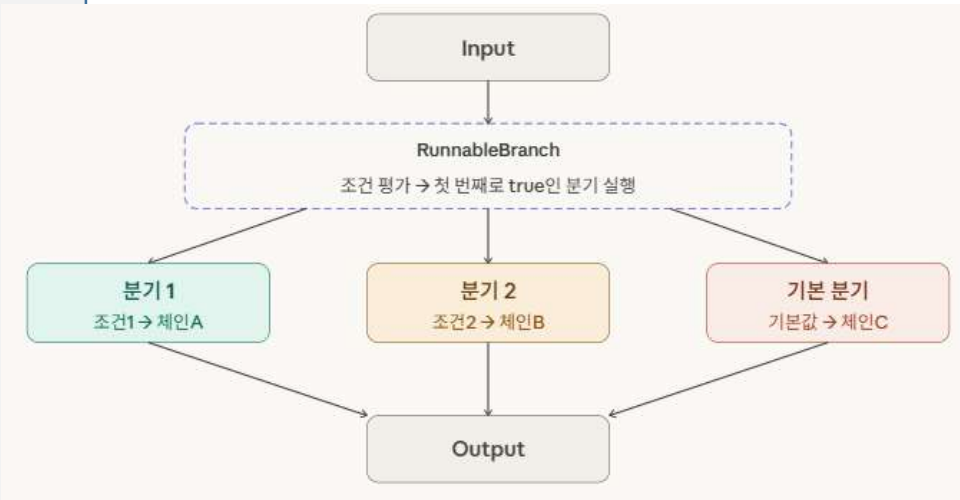
```
const parallelChain = RunnableParallel.from({
  summary: summaryChain,
  sentiment: sentimentChain,
  entities: entityExtractionChain,
  keywords: keywordChain,
});

const result = await parallelChain.invoke(input);
// → { summary: "...", sentiment: "...", entities: [...], keywords: [...] }
```

Chain

Branching Chain (RunnableBranch)

- 입력에 따라 다른 체인으로 라우팅 (조건부 라우팅)

```
// 분기별 프롬프트 체인
const technicalChain = ChatPromptTemplate
  .fromTemplate("Answer this technical question: {question}")
  .pipe(model).pipe(parser);
const creativeChain = ChatPromptTemplate
  .fromTemplate("Be creative and answer: {question}")
  .pipe(model).pipe(parser);
const generalChain = ChatPromptTemplate
  .fromTemplate("Answer helpfully: {question}")
  .pipe(model).pipe(parser);
const router = RunnableBranch.from([
  [    (x) => x.topic === "technical" ,    technicalChain  ],
  [    (x) => x.topic === "creative" ,    creativeChain  ],
  generalChain  // 기본값
]);
const result = await router.invoke({
  topic: "technical",
  question: "What is Big O notation?"
});
```



Chain

Multi-Chain

- 복합 워크플로우 (RunnableSequence, RunnableParallel, RunnableBranch, RunnablePassthrough, RunnableLambda조합)

```
const appChain = RunnableSequence.from([
  // 입력 전처리
  new RunnableLambda({ func: preprocess }),
  // 병렬 처리
  RunnableParallel.from({
    rag: ragChain,
    search: webSearchChain,
    memory: memoryRecallChain,
  }),
  // 결과 합치기 & 라우팅
  combineAndRouteChain,
  // 최종 생성
  finalGenerationChain,
  // 출력 정리
  new StringOutputParser(),
]);
```

Chain

RunnableParallel + RunnablePassthrough 사용 예제

- jokeChain으로 농담을 생성한 뒤, 농담(joke), 농담의 길이(length), 원본 입력(original) 을 한 번에 병렬로 반환

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import {   RunnableParallel,  RunnablePassthrough,  RunnableLambda,} from "@langchain/core/runnables";

//모델 정의
const model = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0.7,
});

//프롬프트 정의
const prompt = ChatPromptTemplate.fromTemplate("{input}");

//체인
const jokeChain = prompt.pipe(model).pipe(new StringOutputParser());
```

Chain

RunnableParallel + RunnablePassthrough 사용 예제

```
//길이 계산 체인
const lengthChain = new RunnableLambda({
  func: async (text: string) => `답변 길이: ${text.length}자`
});

//병렬 실행
const overall = jokeChain.pipe(
  RunnableParallel.from({
    joke: new RunnablePassthrough(),
    length: lengthChain,
    original: new RunnablePassthrough(),   // 입력 그대로 전달
  })
);

async function main() {
  const result = await overall.invoke({
    input: "웃긴 농담 하나 해줘",
  });
  console.log(result);
}
main();
```

RunnableLambda : LangChain에서 일반 JavaScript 함수를 Runnable로 만들어주는 클래스

RunnablePassthrough()는 입력값을 그대로 통과시키는 특수 Runnable
jokeChain에서 나온 농담 텍스트를 그대로 "joke" 키에 넣어 줌

RunnableParallel 안에서 RunnablePassthrough()는 이전 단계의 입력이 아니라, jokeChain의 출력을 받음

C:₩langchain_dev>npx tsx runnable_example.ts

Parser

출력 파서 (Output Parser)

Parser

- @langchain/core/output_parsers

Parser	출력 타입	특징 &용도
StringOutputParser	string	가장 기본적. 텍스트 그대로 추출 .pipe(model).pipe(new StringOutputParser()) LLM 출력에서 텍스트만 받고 싶을 때 JSON 구조화가 필요 없을 때 검증, 포맷 검사, 스키마 없음
JsonOutputParser	any / object	LLM이 JSON 문자열을 출력했다고 가정하고 파싱 .pipe(model).pipe(new JsonOutputParser())
StructuredOutputParser	Zod로 정의한 타입	Zod 스키마 기반으로 구조화된 객체 강제 파싱 스키마 검증, 타입 안정성 model.withStructuredOutput(schema)
OutputFixingParser	원래 parser의 출력	다른 parser가 실패했을 때 LLM으로 자동 수정 OutputFixingParser.fromLLM(llm, baseParser)
BytesOutputParser	Uint8Array 등	binary 데이터 (이미지, 파일 등) 그대로 통과 바이너리 생성 모델 chain 끝에 붙임

출력 파서 (Output Parser)

StringOutputParser

- LLM이 반환하는 AIMessage 객체에서 텍스트 내용(content)만 추출하여 문자열(string)로 반환하는 파서

```
// 프롬프트 정의
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "당신은 친절한 AI 어시스턴트입니다."],
  ["human", "{question}"],
]);

// 모델 초기화
const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0.7,
 maxTokens: 200,
});

// 체인 구성 (LCEL 방식)
const chain = prompt.pipe(llm).pipe(new StringOutputParser());

const result = await chain.invoke({
  question: "파이썬의 장점 3가지를 알려주세요.",
})
```

```
C:₩langchain_dev>npx tsx stringparser_example.ts
```

출력 파서 (Output Parser)

StringOutputParser

- 쉼표로 구분된 항목을 리스트로 변환

```
const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0,
});

const prompt = ChatPromptTemplate.fromMessages([
  ["system", "항목을 쉼표로 구분된 리스트로 작성하세요."],
  ["human", "{topic}에 대한 3가지 예시를 알려주세요."]
]);

const chain = prompt.pipe(llm).pipe(new StringOutputParser());

const response = await chain.invoke({
  topic: "과일"
});

// 쉼표 기준으로 리스트 변환
const list = response.split(",").map(item => item.trim());
```

```
C:₩langchain_dev>npx tsx commaparser.ts
```

출력 파서 (Output Parser)

StructuredOutputParser

- LLM의 텍스트 출력을 **JSON 객체로 파싱**

```
import { z } from "zod";
import { StructuredOutputParser } from "@langchain/core/output_parsers";
//  Zod 스키마 정의 (Pydantic 대신 사용)
const BookInfoSchema = z.object({
  title: z.string().describe("책 제목"),
  author: z.string().describe("저자 이름"),
  year: z.number().describe("출판 연도"),
  genre: z.string().describe("장르"),
});
// JSON 파서 생성
const parser = StructuredOutputParser.fromZodSchema(BookInfoSchema);
// 포맷 지시사항 확인
console.log(parser.getFormatInstructions());

const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0,
  maxCompletionTokens: 200,
});
```

```
C:₩langchain_dev>npx tsx structuredparser.ts
```

출력 파서 (Output Parser)

Zod 스키마

- TypeScript 환경에서 스키마 선언 & 유효성 검사 라이브러리
- 런타임 데이터 검증 (Runtime Validation)
- 타입스크립트 타입 자동 추론 (Static Type Inference)
- 직관적이고 체이닝 가능한 API
- 복잡한 중첩 객체, 배열, 유니온, discriminated union 등을 쉽게 정의 가능
- 에러 메시지 커스터마이징이 편리
- 가볍고 성능이 좋음
- Tool 입력 정의, OutputParser, Structured Output 등에 매우 많이 사용됨

```
npm install zod
```

출력 파서 (Output Parser)

Zod 스키마 메서드

종류	예시	설명
기본 타입	z.string(), z.number(), z.boolean()	원시 타입
문자열 검증	.min(5), .max(100), .email(), .url()	길이, 형식 검사
숫자 검증	.int(), .positive(), .min(0)	정수, 범위 검사
객체	z.object({ ... })	객체 스키마
배열	z.array(z.string())	배열
선택적/기본값	.optional(), .default(value)	optional, default
열거형	z.enum(["A", "B", "C"])	union of literals
유니온	z.union([z.string(), z.number()])	여러 타입 허용
Discriminated Union	z.discriminatedUnion("type", [...])	타입에 따른 분기 (매우 강력)
변환	.transform((val) => val.toUpperCase())	값 변환

출력 파서 (Output Parser)

Zod 스키마 메서드

```
import { z } from "zod";

// 1. 스키마 정의
const UserSchema = z.object({
  id: z.number().int().positive(),
  username: z.string().min(3, "사용자 이름은 최소 3자 이상이어야 합니다").max(20),
  email: z.string().email("올바른 이메일 형식이 아닙니다"),
  age: z.number().min(18, "18세 이상만 가입 가능합니다").optional(), // 선택적 필드
  isActive: z.boolean().default(true),
  tags: z.array(z.string()).min(1),            // 문자열 배열
  role: z.enum(["user", "admin", "moderator"]), // 열거형
});

// 2. 타입 자동 추론
type User = z.infer<typeof UserSchema>;   // ← TypeScript 타입 자동 생성

// 3. 데이터 검증
const rawData = {
  id: 1,
  username: "giyeon",
```

출력 파서 (Output Parser)

Zod 스키마 메서드

```
  email: "giyeon@example.com",
  age: 25,
  tags: ["developer", "typescript"],
  role: "user" as const,
};

try {
  const user = UserSchema.parse(rawData);   // 검증 성공 시 데이터 반환
  console.log(user);
} catch (error) {
  if (error instanceof z.ZodError) {
    console.log(error.errors);   // 상세 에러 정보
  }
}

// 안전하게 검증하고 싶을 때 (에러를 throw하지 않음)
const result = UserSchema.safeParse(rawData);
if (result.success) {
  console.log(result.data);
} else {
  console.log(result.error);
}
```

출력 파서 (Output Parser)

BytesOutputParser

- LLM의 응답을 문자열이 아닌 Uint8Array(바이트 데이터) 형태로 반환하는 Output Parser

```
const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0,
  maxCompletionTokens: 200,
});
// 프롬프트 정의
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "당신은 간단한 설명을 작성하는 AI입니다."],
  ["human", "{topic}에 대해 한 문장으로 설명하세요."]
]);
// 체인 구성
const chain = prompt.pipe(llm).pipe(new BytesOutputParser());

// 실행
const result = await chain.invoke({
  topic: "인공지능"
});
```

```
C:₩langchain_dev>npx tsx bytes_output_example.ts
```

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

- 기존의 대규모 언어 모델에 검색 기능을 추가하여, 주어진 질문이나 문제에 대해 더 정확하고 풍부한 정보를 기반으로 답변을 생성
- 모델이 학습 데이터에 포함되지 않은 외부 데이터를 실시간으로 검색(retrieval)하고, 이를 바탕으로 답변을 생성(generation)하는 과정을 포함
- 환각(생성된 내용이 사실이 아닌 것으로 오인되는 현상)을 방지하고, 모델이 최신 정보를 반영하거나 더 넓은 지식을 활용할 수 있게 합니다.
- RAG(Retrieval-Augmented Generation) 파이프라인
  - Document Loading : 외부 데이터 소스에서 정보를 수집하고, 필요한 형식으로 변환하여 시스템에 로드합니다.
  - Text Splitting : 불러온 데이터를 작은 크기의 단위(chunk)로 분할하는 과정
  - Embedding → chunk를 벡터로 변환
  - Vector Store → 벡터 저장 & 검색 가능하게 함
  - Retriever → 쿼리에 대해 관련 chunk 검색 (top-k)
  - Prompt + Retrieved Context → LLM에 컨텍스트 + 질문 주입
  - Generation → 최종 답변 생성

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

```
[Document Loader]
        ↓
[Text Splitter]
        ↓
[Embedding Model]
        ↓
[Vector Store]
        ↓
[Retriever]
        ↓
[Prompt Template]
        ↓
[LLM]
        ↓
[Answer]
```

```
// 1. 문서 로딩
const docs = await loader.load();

// 2. 청킹
const chunks = await splitter.splitDocuments(docs);

// 3. 벡터 저장
const vectorStore = await MemoryVectorStore.fromDocuments(chunks, embeddings);

// 4. 검색
const retriever = vectorStore.asRetriever();

// 5. 질문
const docsRetrieved = await retriever.invoke(question);

// 6. context 생성
const context = docsRetrieved.map(d => d.pageContent).join("₩n");

// 7. LLM 호출
const response = await model.invoke(prompt);
```

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

구분	전통적 LLM	RAG 시스템
지식 소스	고정된 학습 데이터	동적 외부 데이터베이스
정보 신선도	학습 시점 기준	실시간 반영 가능
운용 비용	주기적 재학습	데이터 업데이트만으로 충분
투명성	내부 매개변수에 의존	출처 표기 가능
처리 범위	토큰 제한 내에서만 작동	외부 저장소 확장 가능

RAG (Retrieval-Augmented Generation)

RAG 주요 컴포넌트와 클래스

단계	클래스	설명/구현체
Document Loader	PDFLoader, WebBaseLoader, TextLoader 등 @langchain/community/document_loaders/fs 등	PDFLoader.fromPath(), WebBaseLoader.fromUrls()
Text Splitter	RecursiveCharacterTextSplitter @langchain/core/text_splitter	chunkSize=1000, chunkOverlap=200 추천
Embeddings	OpenAIEmbeddings, HuggingFaceInferenceEmbeddings 등 @langchain/openai, @langchain/community	OpenAIEmbeddings({ model: "text-embedding-3-small" })
Vector	StoreFAISS, Chroma, SupabaseVectorStore, Pinecone 등 @langchain/community/vectorstores/*	FAISS.fromDocuments() (로컬 최고), SupabaseVectorStore (프로덕션)
Retriever	vectorStore.asRetriever({ k: 4 }) @langchain/core/retrievers	.asRetriever() → 가장 간단. MMR, SelfQueryRetriever 등 고급 retriever 도 지원
RAG Chain	createRetrievalChain (또는 직접 LCEL) @langchain/core/chains 또는 LCEL	createRetrievalChain({ retriever, combineDocsChain })

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

```
import "dotenv/config";
import { FaissStore } from "@langchain/community/vectorstores/faiss";
import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import { OpenAIEmbeddings, ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";

// 1. Load
const loader = new PDFLoader("company_policy.pdf");
const docs = await loader.load();

// 2. Split
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});
const chunks = await splitter.splitDocuments(docs);
// 3. Embedding
const embeddings = new OpenAIEmbeddings({
  model: "text-embedding-3-small",
});
```

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

```
// 3. Embedding
const embeddings = new OpenAIEmbeddings({
  model: "text-embedding-3-small",
});

const vectorStore = await FaissStore.fromDocuments(
  chunks,
  embeddings
);
// 4. Retriever  - 유사한 문서 5개 찾을 것
const retriever = vectorStore.asRetriever({ k: 5 });

// 5. Prompt
const prompt = ChatPromptTemplate.fromMessages([
  ["system", `다음 컨텍스트만 사용해서 답변해. 모르면 "모르겠어요"라고 말해.
{context}`],
  ["human", "{input}"],
]);
// 6. Model
const model = new ChatOpenAI({ model: "gpt-4o-mini" });
```

```
npm install faiss-node
npm uninstall pdf-parse
npm install pdf-parse@1 --legacy-peer-deps
npm list pdf-parse
```

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

```
// 7. 실행 (직접 구성 ☆ 핵심)
const question = "연차 휴가 규정은 어떻게 되나요?";

// retriever 실행
const docsRetrieved = await retriever.invoke(question);

// context 문자열로 변환
const context = docsRetrieved.map(d => d.pageContent).join("₩n₩n");

// prompt 생성
const finalPrompt = await prompt.invoke({
  context,
  input: question,
});

// LLM 호출
const response = await model.invoke(finalPrompt);
console.log(response.content);
```

```
C:₩langchain_dev>npx tsx rag_example.ts
```

RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation)

- 대화형 RAG (Conversational RAG) - 이전 대화 기록(chat history)을 기억하면서, 사용자의 질문(input)에 대해 관련 문서를 검색(retriever) 한 후 답변하는 체인

```
// 프롬프트에 chat_history placeholder 추가
const conversationalPrompt = ChatPromptTemplate.fromMessages([
  ["system", "너는 회사 정책 전문가야.₩n컨텍스트: {context}"],
  new MessagesPlaceholder("chat_history"),
  ["human", "{input}"],
]);
const ragChainWithHistory = new RunnableWithMessageHistory({
  runnable: RunnableSequence.from([
    { input : new RunnablePassthrough(),
      chat_history : new RunnablePassthrough(),
      context: retriever,
    },
    conversationalPrompt,
    model,
  ]),
  getMessageHistory: async (sessionId) =>    new ChatMessageHistory({ sessionId }),
  inputMessagesKey: "input",
  historyMessagesKey: "chat_history",
});
```

RunnableWithMessageHistory : LangChain에서 대화 기록을 자동으로 관리해주는 래퍼 클래스

ChatMessageHistory: 실제 대화 기록을 저장하는 가장 기본적인 메모리 클래스

RAG (Retrieval-Augmented Generation)

Load Data

- CheerioWebBaseLoader 클래스를 사용하여 웹페이지의 텍스트 데이터를 추출하여 Document 객체의 리스트로 변환합니다. (HTML 파싱, selector로 원하는 부분만 추출 가능)

```
// 1. 문서 로딩 (웹페이지)
const loader = new CheerioWebBaseLoader(
  "https://ko.wikipedia.org/wiki/%EC%9C%A4%EB%8F%99%EC%A3%BC"   // ← 분석할 웹페이지 URL
);
const docs = await loader.load();
// 2. 청킹
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 500,
  chunkOverlap: 100,
});
const chunks = await splitter.splitDocuments(docs);
// 3. 벡터 저장
const embeddings = new OpenAIEmbeddings({
  model: "text-embedding-3-small",
});
const vectorStore = await FaissStore.fromDocuments(
  chunks,
  embeddings
);
```

```
C:₩langchain_dev>npx tsx rag_example2.ts
```

RAG (Retrieval-Augmented Generation)

Text Split

- 자연어 처리(NLP) 기술을 활용하여 큰 문서를 처리가 쉽도록 문단, 문장 또는 구 단위로 나누는 작업
- RecursiveCharacterTextSplitter

```
// docs는 이전 단계(WebBaseLoader 등)에서 생성된 Document[]
async function main() {
  console.log("₩nText Split 시작₩n");
  const textSplitter = new RecursiveCharacterTextSplitter({
    chunkSize: 1000,
    chunkOverlap: 200,
  });
  // Documents → 작은 chunk Documents
  const splits = await textSplitter.splitDocuments(docs);
  console.log("chunk 개수:", splits.length);
  console.log("₩n10번째 chunk 전체 객체₩n");
  console.log(splits[10]);
  console.log("₩n10번째 chunk 텍스트₩n");
  console.log(splits[10].pageContent);
  console.log("₩n10번째 chunk metadata₩n");
  console.log(splits[10].metadata);
}
```

RAG (Retrieval-Augmented Generation)

Indexing

- 검색 시간을 단축시키고, 검색의 정확도를 높이는 데 중요한 역할을 합니다.
- LangChain 라이브러리를 사용하여 텍스트를 Embedding(Vector 생성)으로 변환하고, vectorStore에 저장되면서 indexing이 내부적으로 수행, 저장된 Embedding을 기반으로 유사성 검색을 수행

```
const embeddings = new OpenAIEmbeddings({
  model: "text-embedding-3-small",
});
// Documents → Embedding → Chroma 저장
const vectorStore = await Chroma.fromDocuments(
  splits,          // Text Split 단계에서 생성된 Document[]
  embeddings,
  {
    collectionName: "wiki_docs",
  }
);
// 유사도 검색
const docs = await vectorStore.similaritySearch(
  "격하 과정에 대해서 설명해주세요.",
  4
);
console.log("검색된 문서 개수:", docs.length);
```

RAG (Retrieval-Augmented Generation)

Retrieval & Generation

- 사용자의 입력을 바탕으로 쿼리를 생성하고, 인덱싱된 데이터에서 가장 관련성 높은 정보를 검색합니다
- Retriever (VectorStoreRetriever 등) : invoke() 시 질문을 받아 Document[] 반환
- vectorstore.as_retriever() 는 Chroma 벡터 스토어를 검색기로 사용하여 사용자의 질문과 관련된 문서를 검색
- format_docs()는 검색된 문서들을 하나의 문자열로 반환합니다.

```
const model = new ChatOpenAI({  model: "gpt-4o-mini",  temperature: 0, });
// Retriever (Chroma VectorStore에서 생성)
const retriever = vectorStore.asRetriever();
// 검색된 문서들을 하나의 문자열로 변환
function formatDocs(docs) {
  return docs.map((doc) => doc.pageContent).join("₩n₩n");
}
// RAG Chain
const ragChain = {
  context: retriever.pipe(formatDocs),
  question: new RunnablePassthrough(),
}
  .pipe(prompt)
  .pipe(model)
  .pipe(new StringOutputParser());
const result = await ragChain.invoke(  "격하 과정에 대해서 설명해주세요.");
```

RAG (Retrieval-Augmented Generation)

Document Loader

- LangChain.js에서 외부 데이터를 LangChain의 표준 Document 객체로 변환해주는 컴포넌트
- 모든 Loader는 RAG 파이프라인의 첫 번째 단계에서 사용되며, 다양한 형식의 파일·웹페이지·데이터베이스 등을 불러와서 이후 Text Splitter → Embeddings → Vector Store 단계로 넘겨줍니다.
- 모든 Loader는 BaseDocumentLoader (또는 BaseLoader) 인터페이스를 구현
- Loader는 Runnable이므로 .pipe()로 체이닝 가능
- .load() - 모든 문서를 한 번에 메모리에 로드 (단일파일에 사용)
- .lazyLoad() - 제너레이터 방식으로 하나씩 로드 (대용량 파일/디렉토리 처리에 메모리 효율적)
- Document[] 배열 반환
- Document.pageContent: 실제 텍스트 내용 (string)
- Document.metadata: 문서에 대한 부가 정보 (객체) - 파일명, 페이지 번호, URL, 생성일 등

RAG (Retrieval-Augmented Generation)

Document Loader

Loader 클래스	설명
TextLoader	텍스트 파일의 내용을 랭체인의 Document 객체로 변환하고 이를 리스트 형태로 반환 파일 경로를 인자로 받아 해당 파일의 내용을 불러온 후, load 메소드를 통해 파일의 내용을 담고 있는 Document 객체로 변환 일반 .txt 파일 로드 인코딩 지정 가능 (utf-8)
PDFLoader	PDF 파일 로드 페이지 단위로 분리 가능 (splitPages: true) import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf"
CSVLoader	CSV 파일 로드 각 행을 하나의 Document로 변환. 컬럼 지정 가능
DocxLoader	Word 문서 (.docx) 로드 MS Office 문서 지원
JSONLoader	JSON 파일 로드 특정 경로(jq_schema) 지정 가능

RAG (Retrieval-Augmented Generation)

Document Loader

Loader 클래스	설명
CheerioWebBaseLoader	특정 웹 페이지의 내용을 로드하고 파싱하기 위해 설계된 클래스 web_paths 매개변수 : 로드할 웹 페이지의 URL을 단일 문자열 또는 여러 개의 URL을 시퀀스 배열로 설정 bs_kwargs 매개변수 : BeautifulSoup을 사용하여 HTML을 파싱할 때 사용되는 인자들을 딕셔너리 형태로 제공 bs4.SoupStrainer를 사용하여 특정 클래스 이름을 가진 HTML 요소만 파싱할 수 있습니다 JavaScript 실행 안 함 (정적 페이지에 좋음)
PlaywrightWebBaseLoader	동적 웹페이지 로드JavaScript 실행 지원 (SPA, 로그인 페이지 등)
UnstructuredLoader	복잡한 파일 (PDF, DOCX 등) 고급 파싱테이블, 레이아웃 잘 인식 (무거움)
NotionLoader, SlackLoader 등	SaaS 서비스 연동API 키 필요

RAG (Retrieval-Augmented Generation)

Document Loader

- CSVLoader 클래스 : CSV 파일에서 데이터를 로드, 각 행을 추출하여 서로 다른 Document 객체로 변환
- Document 객체의 리스트 형태로 반환
- CSV 관련 설정을 csv_args 매개변수를 사용하여 설정

```
// 1. 문서 로딩 (CSV)
const loader = new CSVLoader("policy.csv");
const docs = await loader.load();
// 2. 청킹
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 300,
  chunkOverlap: 50,
});
const chunks = await splitter.splitDocuments(docs);
// 3. 벡터 저장
const embeddings = new OpenAIEmbeddings({
  model: "text-embedding-3-small",
});
const vectorStore = await FaissStore.fromDocuments(
  chunks,
  embeddings
);
```

```
C:₩langchain_dev>npx tsx rag_example3.ts
```

RAG (Retrieval-Augmented Generation)

Document Loader

- PyPDFLoader : PDF 파일에서 텍스트를 추출

```
!pip install -q pypdf
```

```
from langchain_community.document_loaders import PyPDFLoader

pdf_filepath = '000660_SK_2023.pdf'
loader = PyPDFLoader(pdf_filepath)
pages = loader.load()

print(len(pages))
print(data[0])
```

Tool

Tool

Tool

- LLM(특히 ChatModel)이 외부 기능(검색, 계산, API 호출, DB 조회 등)을 사용할 수 있게 해주는 핵심 컴포넌트
- "생각 → 도구 선택 → 실행 → 관찰 → 생각" 의 Agent / ReAct(Reasoning + Acting) 패턴 구현
- 활용 : 외부 API 호출, 계산 작업 수행, 데이터 검색, 자동화 작업

ReAct(Reasoning + Acting) 패턴 :
LLM이 생각(Reasoning) 하고 행동(Acting, Tool 사용) 하면서 문제를 해결하는 방식
질문 → 생각(Reason) → 행동(Action: Tool 호출) → 관찰(Observation) → 반복 → 최종 답변

질문: "서울 날씨 + 3일 뒤 온도 합계는?"

Thought: 서울 날씨 조회 필요
Action: weather API 호출
Observation: 20도

Thought: 3일 뒤 온도 필요
Action: weather API 호출
Observation: 25도

Thought: 계산 필요
Action: calculator 호출

Final Answer: 45도

Tool

langChain 내장 Tool

- TavilySearchResults : 최신 정보 검색, 요약된 답변 + 정제, 완전 실시간 데이터 아님,RAG / Agent에 최적화, (1000/month)
- DuckDuckGoSearch : 공식 무료 검색 인터페이스를 제공, 인증(API Key) 없이도 검색 가능, 요청 속도 제한 (rate limit) , 검색 결과 품질 제한 (Google 대비)
- SerpAPI : Google / Bing 결과를 그대로 JSON으로 제공, 정확도 매우 높음 (구글 기반) , 유료
- Calculator : 간단한 수학 계산 전용 Tool, LLM이 계산 실수를 하지 않도록 보조
- Python REPL : Python 코드 실행 환경 제공, 데이터 분석, 반복문 / 로직 처리
- Retriver : 벡터DB에서 관련 문서 검색, LLM에게 지식 공급, FAQ 챗봇, 사내 문서 검색, RAG (Retrieval-Augmented Generation)
- DynamicTool : 커스텀 Tool, 함수 기반 Tool, 사내 API 호출, DB 조회, 특정 비즈니스 로직 호출 목적
- Browser : 웹 검색 + 페이지 내용 가져오기 , 최신 정보 검색, 뉴스 / 트렌드, 웹 페이지 요약

https://serpapi.com/account-api?utm_source=chatgpt.com
https://app.tavily.com/home

Tool

Tool 관련 클래스 & 모듈

클래스 / 함수	패키지	특징 &용도
tool	@langchain/core/tools	가장 간단한 Tool 생성 함수 (데코레이터 스타일)
StructuredTool	@langchain/core/tools	Zod 스키마 기반 구조화 Tool의 베이스 클래스
DynamicStructuredTool	@langchain/core/tools	런타임에 스키마 동적 생성 (JSON Schema도 지원)
ToolNode	@langchain/langgraph/prebuilt	LangGraph에서 Tool들을 실행하는 노드 (Agent용)
AIMessage.tool_calls	@langchain/core/messages	모델이 Tool 호출 요청 시 여기에 배열로 저장
ToolMessage	@langchain/core/messages	Tool 실행 결과를 모델에게 되돌려줄 때 사용 (tool_call_id 필수)

Tool

Tool을 모델에 연결하는 주요 파라미터 / 메서드

메서드 / 파라미터	설명
model.bindTools(tools)	모델에 Tool 목록 바인딩 → tool calling 가능하게 함 const modelWithTools = model.bindTools([calculator, searchTool]);
strict	OpenAI 계열에서 JSON Schema strict 모드 .bindTools(tools, { strict: true })
withStructuredOutput()	Tool calling이 아닌 구조화 출력 강제 (단순 출력용) model.withStructuredOutput(z.object({...}))
createToolCallingAgent ()	Tool calling 기반 Agent 생성 createToolCallingAgent({ llm: modelWithTools, tools, prompt })
AgentExecutor	Agent 실행기 new AgentExecutor({ agent, tools })

Tool

Tool calling 시 Message 흐름

1. 모델 → AIMessage (tool_calls: [{id: "call_abc", name: "calc", args: {...}}])
2. 외부에서 tool 실행 → 결과 얻음
3. ToolMessage 생성 (tool_call_id 필수로 매칭!)
4. messages 배열에 AIMessage + ToolMessage 추가
5. 다시 모델 호출 → 이제 모델은 결과를 알고 최종 답변 생성

Tool

Tool 정의

- TavilySearch는 llm.bindTools([tool])로 LLM에 바인딩하고, tool.batch(aiMsg.tool_calls)로 실제 검색을 실행한 뒤 결과를 다시 LLM에 전달하는 패턴을 사용합니다.

```
사용자 질문
→  prompt.pipe(llmWithTools).invoke()
    LLM이 "Tavily로 검색해야겠다" 판단 → tool_call 생성
→ tavilyTool.batch(aiMsg.tool_calls)
    실제 웹 검색 실행 → ToolMessage[] 반환
→ chain.invoke({ messages: [질문, aiMsg, ...toolMessages] })
    검색 결과 + 원래 질문으로 LLM 재호출 → 최종 한국어 답변
```

```
C:₩langchain_dev>npx tsx  use_searchtool.ts
C:₩langchain_dev>npx tsx  searchtoolbind.ts
```

Tool

Tool 정의

- Agent는 LLM 기반의 의사결정 엔진으로 사용자 질문 이해 → 어떤 작업이 필요한지 판단 → 필요한 Tool 선택 → Tool 실행 결과 해석 → 최종 답변 생성 합니다
- Agent = LLM + Reasoning + Tool Selection
- Agent가 ReAct 패턴(Reasoning + Acting)을 사용합니다.
- Agent는 필요할 때만 사용 (동적 의사결정)
- Agent는 다중 Tool 사용 가능
- Tool 결과를 다시 Reasoning (Tool 결과를 다시 Reasoning)

Tool 정의 → LLM 바인딩 → Agent 실행
1. add_numbers
2. multiply_numbers
3. web_search
4. get_current_time

npm install @langchain/openai @langchain/core @langchain/langgraph @langchain/tavily zod dotenv

C:₩langchain_dev>npx tsx agent_usetool.ts

Tool

Tool

RAG는 고정 파이프라인 구조이며, Tool 사용이 제한적이고
검색 중심이기 때문에 Retriever는 필수이다

1. 문서 로딩
2. Chunking
3. 벡터 저장
4. 질문 입력
5. LLM 판단
   → 벡터 검색?
   → Tool 호출?

6-A. (RAG 선택 시)
   → Retriever 검색
   → Context 생성

6-B. (Tool 선택 시)
   → Tool 실행 (검색, 계산, API 등)

7. 결과 통합
8. LLM 최종 응답 생성

Tool

Tool

1. 문서 로딩
2. Chunking
3. 벡터 저장

4. 질문 입력

5. Agent 실행
6. LLM 판단 (Planning)
   → 어떤 Tool 쓸지 결정
7. Tool 실행
   → Retriever (RAG)
   → Search API
   → DB 조회
   → 계산
8. 결과 Observation
9. 필요 시 반복 (ReAct loop)
10. 최종 LLM 응답 생성

Agent +Tool은 동적 의사결정 중심의 구조이며, Tool을 적극적으로 사용하고, Retriever사용은 선택적이다

Tool

Tool 정의

- Agent가 Tool을 사용할 때 입력 파라미터 구조를 정의할 때 StructuredTool + zod를 사용합니다
- StructuredTool : 입력 스키마가 정의된 Tool
- StructuredTool.fromFunction() : 기존 JavaScript 함수를 LangChain Tool로 변환하는 helper
- StructuredTool.fromFunction() 과 StructuredTool + zod 는 Agent가 Tool을 안전하게 호출하도록 만드는 구조입니다

```
StructuredTool.fromFunction({
    "name": "...",                //Tool 이름
    "description": "...",         // Tool 설명(Agent 읽음)
     schema: z.object(...),       // 입력 파라미터 구조
     func: async (input) => {}    // 실제 실행 함수
    }
}
```

Memory와 대화 관리

Memory와 대화 관리

메모리의 역할 & 유형

- 대화 기록 저장하고 관리
- 대화의 흐름과 맥락(컨텍스트) 보존
- 사용자별 대화 세션 관리(개인화)
- 토큰 사용량 최적화를 위한 메모리 관리
- LLM은 stateless(상태 없음) → 매번 새로 호출되면 이전 대화를 잊어버림.
- Memory를 통해 대화 이력(chat history)을 저장·관리·주입하여 자연스러운 멀티턴 대화 구현
- LCEL(LangChain Expression Language) 기반의 RunnableWithMessageHistory + BaseChatMessageHistory 구현체를 사용하는 방식

```
const chainWithHistory = new RunnableWithMessageHistory({
  runnable: chain,                                  // prompt.pipe(model).pipe(parser) 같은 LCEL chain
  getMessageHistory: async (sessionId) => {
    // sessionId별로 ChatMessageHistory 인스턴스 반환 (또는 DB 로드)
    return new ChatMessageHistory();                // 또는 Redis, Upstash, Supabase 등
  },
  inputMessagesKey: "input",                        // 프롬프트에서 사용자 입력 키
  historyMessagesKey: "chat_history",               // 프롬프트에서 이력이 들어갈 placeholder 키
});
```

Memory와 대화 관리

메모리의 유형

Short-term Memory (단기 메모리)	Long-term Memory (장기 메모리)
• 현재 진행 중인 하나의 대화 스레드(thread) 또는 세션 내에서만 유지되는 메모리 • 같은 대화 안에서 이전 턴의 메시지, tool call 결과, 중간 추론 등을 기억합니다. • Thread-scoped (스레드 단위로 격리됨 → 다른 사용자/다른 대화와 섞이지 않음) • Checkpoint 기반으로 저장 (LangGraph의 checkpointer 사용) • 대화가 끝나거나 스레드가 종료되면 기본적으로 사라지거나 재개 가능하지만, cross-thread 공유 안 됨 • 주로 메시지 이력(message history) 형태로 관리됨 • 전체 히스토리 - 모든 메시지를 그대로 저장, 짧은 대화, 정확한 맥락 필요 • 윈도우 메모리 - 최근 N개 메시지만 유지, 일반적인 챗봇에 사용 • 요약 메모리 - 이전 대화를 요약하여 저장, 긴 대화, 비용 최적화에 사용 • 토큰 버퍼 - 토큰 수 기준으로 자동 trimming, 토큰 예산 관리	• 여러 대화(스레드)를 넘어 지속적으로 저장되는 메모리. • 사용자 선호도, 과거 사실, 학습된 지식, 피드백 등을 cross-thread / cross-session으로 기억합니다 • Cross-thread & persistent (세션 종료 후에도 남음) • LangGraph의 Store (Memory Store)로 JSON 문서 형태로 저장 • Semantic search 지원 • 사용자별 키-값 또는 네임스페이스 + 키로 관리 • LangMem SDK (추가 라이브러리)로 더 쉽게 추출·관리 가능 • 장기 메모리 (Long-term Memory) : 여러 세션에 걸쳐 유지되는 영구적 메모리 • 사용자 프로필 - 사용자 선호도, 속성 저장 ,개인화된 응답에 사용 • 대화 요약 - 과거 세션 요약 저장, 세션 간 맥락 유지에 사용 • 지식 추출 - 대화에서 핵심 정보 추출, 장기 학습 에이전트에 사용

Memory와 대화 관리

대화 관리 메모리 클래스

- RunnableWithMessageHistory : Runnable(LLM, Chain, Agent)에 메모리를 붙여주는 래퍼
- ChatMessageHistory : 실제 대화 저장소 , in-memory 저장 (기본) , 메시지 배열 구조, LangChain 내부 Message 객체 사용

```
import { RunnableWithMessageHistory } from "@langchain/core/runnables";
import { ChatMessageHistory } from "langchain/stores/message/in_memory";

const store = new Map();
// 세션별 메모리 저장
const getMessageHistory = (sessionId: string) => {
  if (!store.has(sessionId)) {
    store.set(sessionId, new ChatMessageHistory());
  }
  return store.get(sessionId);
};
const chainWithHistory = new RunnableWithMessageHistory({
  runnable: llm,   // 또는 chain, agent
  getMessageHistory,
  inputMessagesKey: "input",
  historyMessagesKey: "history",
});
```

1. 사용자 입력
2. 이전 대화 가져오기
3. prompt에 history 포함
4. LLM 호출
5. 결과를 history에 저장

Memory와 대화 관리

대화 관리 메모리 클래스

```
await chainWithHistory.invoke(
  { input: "안녕? 내 이름은 서울이야~" },
  { configurable: { sessionId: "user1" } }
);

await chainWithHistory.invoke(
  { input: "내 이름 기억해?" },
  { configurable: { sessionId: "user1" } }
);
```

User Input
↓
RunnableWithMessageHistory
↓
Chat History Load : getHistory(sessionId)
↓
add User Message
↓
LLM 호출
↓
Message 저장 : add AI Message
↓
History Update

Memory와 대화 관리

대화 관리 메모리 클래스

- ChatMessageHistory : 대화 전용 저장소로서 대화를 위한 구조 (Conversation Memory) , 메시지 타입 존재 (Human / AI / System) , 순서 중요 (대화 흐름 유지), LLM prompt에 바로 사용 가능
- InMemoryStore : 범용 Key-Value 저장소, 타입 제한 없음, 대화 구조 없음, 순서 개념 없음, 상태 / 캐시 / 사용자 정보등 데이터 저장을 위한 구조 (State Store)

InMemeory 저장 구조

```
key: "user1_profile"
value: { name: "철수", age: 30 }
```

ChatMessageMemeory 저장 구조

```
[
  HumanMessage("안녕"),
  AIMessage("안녕하세요"),
  HumanMessage("이름 기억해?")
]
```

```
import { InMemoryStore } from "@langchain/langgraph/store/memory";

const store = new InMemoryStore();

// 노드 안에서 long-term memory 사용 예시
async function saveUserPreference(state) {
  await store.put(["user_prefs", state.userId], "favorite_color", { value: "blue" });
  return state;
}

// 검색 예시
const prefs = await store.get(["user_prefs", "user123"], "favorite_color");
```

Memory 와 대화 관리

메모리의 역할 & 유형

- 간단한 챗봇 : RunnableWithMessageHistory 사용
- 복잡한 에이전트 : LangGraph checkpointer 사용
- MessagesPlaceholder(variable_name="history") : 대화 히스토리가 삽입될 위치를 지정
- Placeholder를 통해 이전 대화 내용이 프롬프트에 동적으로 포함됩니다
- session_id를 통해 서로 다른 사용자나 대화 세션을 구분합니다
- 프로세스가 종료되면 데이터가 사라지는 인메모리 방식입니다
- 짧은 대화 → 기본 ChatMessageHistory (전체 저장)
- 긴 대화 → ConversationSummaryBufferMemory 같은 요약 로직을 커스텀으로 구현 (RunnableWithMessageHistory 내부에서 요약 chain 호출)
- 영속성 → @langchain/community 또는 @langchain/redis, @langchain/upstash-redis, @langchain/supabase 등으로 Redis, Supabase, MongoDB 등에 저장
- 슬라이딩 윈도우 → 커스텀 BaseChatMessageHistory 구현체에서 getMessages() 시 최근 k개만 반환

Memory 와 대화 관리

Memory 클래스와 구현체

클래스/타입	패키지	특징
BaseChatMessageHistory	@langchain/core/chat_history	모든 채팅 이력 저장소의 추상 베이스 클래스 (addMessage, getMessages 등)
BaseListChatMessageHistory		내부적으로 BaseMessage[] 형태로 메시지를 관리
ChatMessageHistory	@langchain/core/chat_history	가장 기본적인 in-memory 이력 저장소 (배열로 Human/AI 메시지 보관)
RunnableWithMessageHistory	@langchain/core/runnables	LCEL chain에 Memory를 쉽게 붙이는 wrapper 이전 모든 대화 내용이 자동으로 현재 프롬프트에 포함되어 연속적인 대화가 가능 Chat History 자동 관리 서로 다른 session_id를 사용하여 여러 사용자나 대화 스레드를 독립적으로 관리할 수 있음 복잡한 메모리 관리 로직 없이도 대화형 AI 구현 가능

Memory 와 대화 관리

RunnableWithMessageHistory 주요 파라미터

파라미터	타입	설명
runnable	Runnable	Memory를 붙일 LCEL chain (prompt.pipe(model) 등)
getMessageHistory	(sessionId: string) => Promise<BaseChatMessageHistory>	세션별 이력 저장소 반환 함수 (DB, Redis 등 연결)
inputMessagesKey	string	프롬프트에서 사용자 입력이 들어갈 키 (보통 "input" 또는 "question")
historyMessagesKey	string	프롬프트에서 대화 이력이 들어갈 placeholder 키 (보통 "chat_history")
outputMessagesKey	string	출력 메시지를 이력에 저장할 때 키 (기본 "output")
messageHistoryFactory	() => BaseChatMessageHistory	getMessageHistory의 대안 (세션 ID 없이 단순 factory)

Memory 와 대화 관리

RunnableWithMessageHistory 주요 메서드

메서드	반환 타입	용도
addMessage(message)	Promise<void>	HumanMessage 또는 AIMessage 추가
addUserMessage(text)	Promise<void>	HumanMessage 간단 추가
addAIResponse(text)	Promise<void>	AIMessage 간단 추가
getMessages()	Promise<BaseMessage[]>	저장된 전체 메시지 배열 가져오기
clear()	Promise<void>	이력 초기화
getMessageCount()	Promise<number>	저장된 메시지 수 (일부 구현체)

Memory 와 대화 관리

RunnableWithMessageHistory 실습 코드

```
import { RunnableWithMessageHistory } from "@langchain/core/runnables";
import { ChatMessageHistory } from "@langchain/community/stores/message/in_memory";import
{ ChatPromptTemplate, MessagesPlaceholder } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";

// 1. 프롬프트 (chat_history placeholder 필수)
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "너는 친절한 한국어 선생님이야."],
  new MessagesPlaceholder("chat_history"),
  ["human", "{input}"],
]);

const model = new ChatOpenAI({ model: "gpt-4o-mini" });
const chain = prompt.pipe(model);

// 2. 세션별 이력 저장소 (in-memory 예시, 실제론 Redis 등 사용)
const messageHistories: Record<string, ChatMessageHistory> = {};

const getHistory = async (sessionId: string) => {
  if (!messageHistories[sessionId]) {
    messageHistories[sessionId] = new ChatMessageHistory();
  }
```

Memory 와 대화 관리

RunnableWithMessageHistory 실습 코드

```
  return messageHistories[sessionId];
};

// 3. Memory 붙인 chain
const chainWithMemory = new RunnableWithMessageHistory({
  runnable: chain,
  getMessageHistory: getHistory,
  inputMessagesKey: "input",
  historyMessagesKey: "chat_history",
});

await chainWithMemory.invoke(
  { input: "자음 탈락이 뭐야?" },
  { configurable: { sessionId: "user123" } }
);

await chainWithMemory.invoke(
  { input: "아까 말한 그거 예시 하나만 더 알려줘." },
  { configurable: { sessionId: "user123" } }
);  // 이전 대화 기억함
```

```
C:₩langchain_dev>npx tsx inmemrory_chathistory.ts
```

메모리와 대화 관리

JSON 파일로 저장하는 커스텀 history

- 파일 기반 저장소 : 동시 접근 시 파일 잠금 문제가 발생할 수 있습니다.
  단일 서버 환경에서만 사용을 권장

```
import { BaseListChatMessageHistory } from "@langchain/core/chat_history";
import { BaseMessage, HumanMessage, AIMessage } from "@langchain/core/messages";
import fs from "fs/promises";
import path from "path";

class SimpleFileChatMessageHistory extends BaseListChatMessageHistory {
  private filePath: string;

  constructor(sessionId: string) {
    super();
    this.filePath = path.join(process.cwd(), "chat_histories", `${sessionId}.json`);
  }

  async getMessages(): Promise<BaseMessage[]> {
    try {
      const data = await fs.readFile(this.filePath, "utf-8");
      const raw = JSON.parse(data);
```

메모리와 대화 관리

JSON 파일로 저장하는 커스텀 history

```
return raw.map((m: any) => {
        if (m._getType() === "human") return new HumanMessage(m);
        if (m._getType() === "ai") return new AIMessage(m);
        return new BaseMessage(m);
      });
    } catch {
      return [];
    }
  }
  async addMessage(message: BaseMessage): Promise<void> {
    const messages = await this.getMessages();
    messages.push(message);
    await fs.mkdir(path.dirname(this.filePath), { recursive: true });
    await fs.writeFile(this.filePath, JSON.stringify(messages, null, 2));
  }
  async clear(): Promise<void> {
    await fs.unlink(this.filePath).catch(() => {});
  }
}
// 사용 예
const fileHistory = new SimpleFileChatMessageHistory("user-2026-03");
```

```
C:₩langchain_dev>npx tsx chatbot_json.ts
```

메모리와 대화 관리

RedisChatMessageHistory

```
import { RedisChatMessageHistory } from "@langchain/community/stores/message/redis";
import { ChatOpenAI } from "@langchain/openai";
import { RunnableWithMessageHistory } from "@langchain/core/runnables";
const chatHistory = new RedisChatMessageHistory({
  sessionId: "user-session-abc123",
  url: "redis://localhost:6379",            // 또는 redis:// + password 등
  ttl: 3600,                                // 초 단위 (선택)
});
const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });
const chain = /* your chain or agent */;
const chainWithHistory = new RunnableWithMessageHistory({
  runnable: chain,
  getMessageHistory: async (sessionId) => {
    return chatHistory;  // 또는 동적으로 sessionId별로 생성
  },
  inputMessagesKey: "input",
  historyMessagesKey: "chat_history",
});
const response = await chainWithHistory.invoke(
  { input: "오늘 기분이 어때?" },
  { configurable: { sessionId: "user-session-abc123" } }
);
```

메모리와 대화 관리

PostgresChatMessageHistory

```
import { PostgresChatMessageHistory } from "@langchain/community/stores/message/postgres";
import { Pool } from "pg";  // 또는 pg-promise 등

const pool = new Pool({
  connectionString: "postgres://user:pass@localhost:5432/mydb",
});

const history = new PostgresChatMessageHistory({
  pool,
  sessionId: "conversation-uuid-1234",
  tableName: "chat_history",      // 기본값
  // schema: "public",            // 필요 시
});

const chainWithHistory = new RunnableWithMessageHistory({
  runnable: yourRunnable,
  getMessageHistory: async () => history,
  inputMessagesKey: "question",
  historyMessagesKey: "history",
});
```

Agent

LangChain 1.0 프레임워크 구성

Agent

- 인공지능(AI)에서 환경과 상호작용하며 목표를 달성하기 위해 행동을 수행하는 자율적인 시스템 또는 엔터티
- 주어진 환경에서 데이터를 감지하고, 처리하여 의사결정을 내리며, 그 결과에 따라 행동한다.
- 환경의 상태를 관찰하고, 이를 바탕으로 적절한 행동을 선택하여 환경에 영향을 미치는 사이클을 반복한다.

LangChain 1.0 프레임워크 구성

Agent

- LLM을 추론 엔진으로 사용하여 어떤 행동을 수행 할 지와 그 행동의 입력이 무엇이 되어야 할지를 결정하는 시스템
- LLM + Tool을 결합하여 복잡한 작업을 자율적으로 수행하는 고급 기능
- LangGraph 기반으로 작동하며, 에이전트는 사용자의 메시지를 처리하면서 필요 시 Tool을 호출하고, 결과를 합성합니다.

User Input (Prompt) : 에이전트의 행동을 안내하는 지침
↓
Agent
↓
Tool Router : Agent가 사용할 수 있는 기능들
↓
Tool 실행
↓
Observation
↓
LLM Reasoning
↓
Final Answer

LangChain 1.0 프레임워크 구성

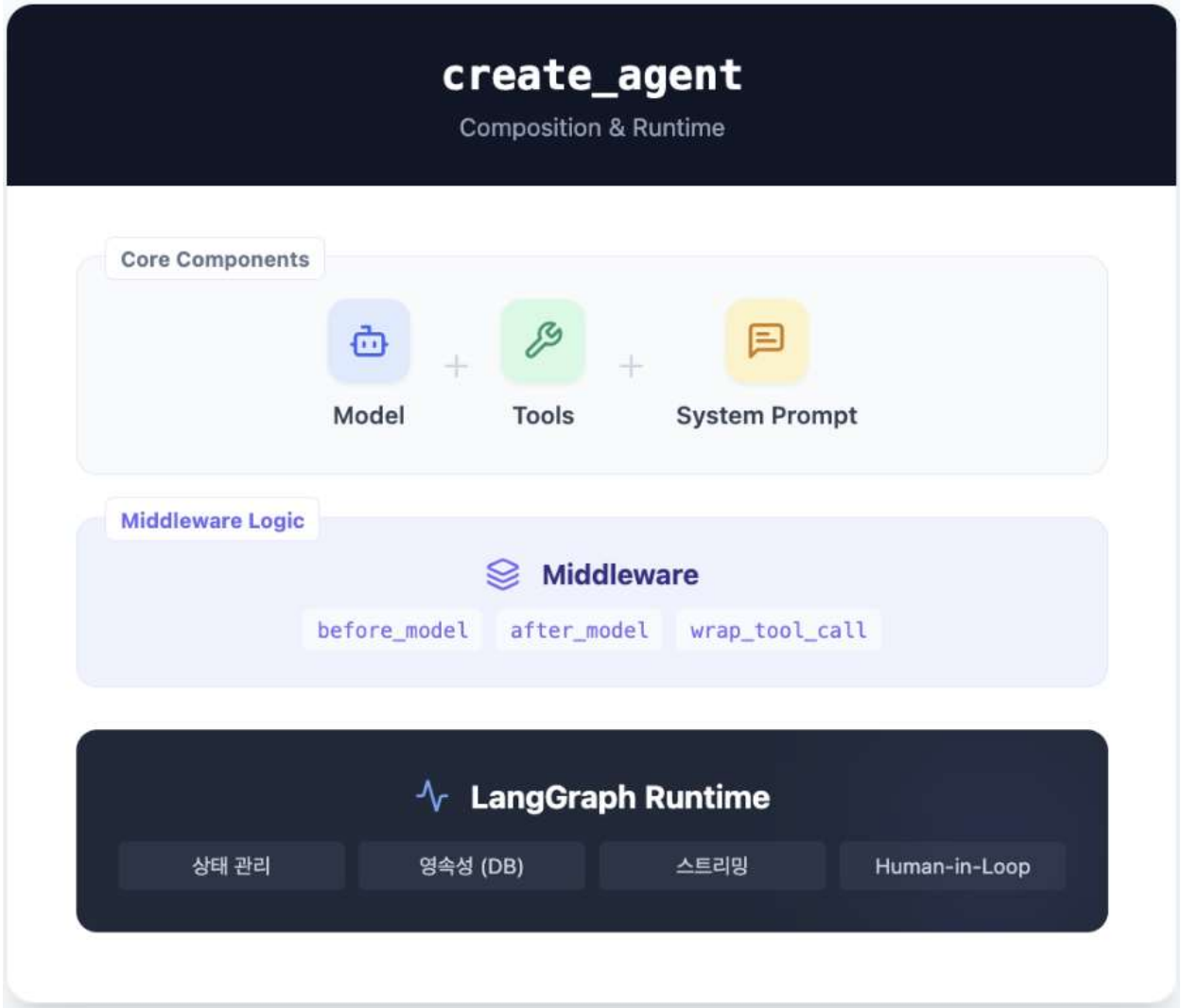
createAgent

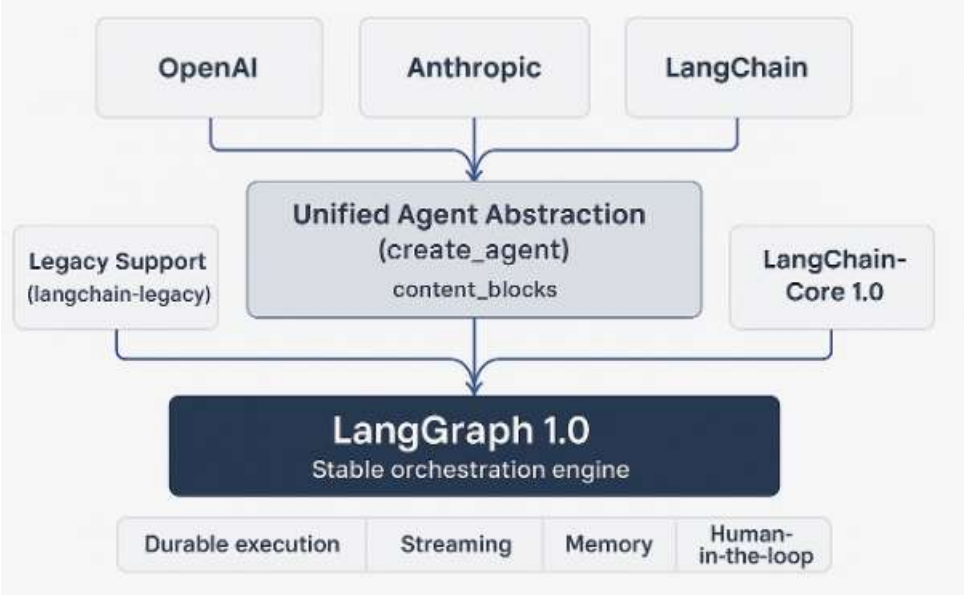
- ReAct 스타일 에이전트 (Reason + Act)를 생산 환경 수준으로 빠르게 만들 수 있게 해줌
- LangGraph 기반의 그래프 런타임 위에서 동작 (체크포인트, human-in-the-loop, rewind, persistence 등 내장)
- 미들웨어를 통해 customizing이 용이함

1. 생각(Thought) : LLM이 현재 상황 보고 다음 행동 결정
2. 행동(Action) : Tool call (tool name + args)
3. 관찰(Observation) : Tool 실행 결과 → ToolMessage로 모델에게 피드백
4. 반복 → 최종 답변(Final Answer) 나올 때까지
5. 종료 조건 : 모델이 tool call 없이 "Final Answer" 형식으로 응답

LangChain 1.0 프레임워크 구성

agent 중심 아키텍처





LangChain 1.0 프레임워크 구성

create_agent의 핵심 파라미터

파라미터	설명
model (필수)	사용할 LLM 모델 (init_chat_model 또는 ChatOpenAI 등) string 또는 BaseChatModel (ChatOpenAI 등) 타입
tools (필수)	에이전트가 사용할 도구 리스트
prompt	시스템 프롬프트 커스터마이징 시 사용(에이전트의 역할/지시사항을 정의) string 또는 SystemMessage 타입
middleware	실행 흐름을 제어하는 미들웨어 리스트 에이전트 루프를 커스터마이징 (프롬프트 캐싱, 라우팅, 검증 등) Array<Middleware> 타입
checkpointer	상태 저장 (대화 히스토리, 중간 상태 persist). 메모리/재개 기능에 필수 BaseCheckpointSaver 타입
maxIterations	무한 루프 방지용
maxTime	전체 실행시간 제한
interruptBefore	특정 노드 전에 중단

LangChain 1.0 프레임워크 구성

create_agent의 핵심 파라미터

파라미터	설명
interruptAfter	특정 노드 후에 중단
recursionLimit	재귀 루프 최대 깊이 제한
Tags/metadata	디버깅 모니터링 용도

```
createAgent({
  model,
  tools,
  prompt: ChatPromptTemplate.fromMessages([
    ["system", "너는 도움이 되는 한국어 AI 어시스턴트야. ..."],
    new MessagesPlaceholder("messages"),
  ]),
  // checkpointer: new MemorySaver() 등
});
```

LangChain 1.0 프레임워크 구성

기본 에이전트 예제

npm install mathjs

```
import "dotenv/config";
import { createAgent,  initChatModel } from "langchain";
import { z } from "zod"
import { tool } from "@langchain/core/tools";
import { DuckDuckGoSearch } from "@langchain/community/tools/duckduckgo_search";
import * as math from "mathjs";
// 1. 도구 정의 (Python의 @tool 데코레이터 → JS에서는 tool 함수 사용)
const searchWeb = tool(
  async ({ query }) => {
    return JSON.stringify({
      answer: `'${query}'에 대한 검색 결과 (모의)`,
      sources: ["https://example.com", "https://mock.data"],
    });
  },
  {
    name: "search_web",
    description: "웹에서 정보를 검색합니다.",
    schema: z.object({
      query: z.string().describe("검색할 키워드나 질문"),
    }),
  }
);
```

LangChain 1.0 프레임워크 구성

기본 에이전트 예제

```
const calculate = tool(
  async ({ expression }) => {
    try {
      const result = math.evaluate(expression);
      return String(result);
    } catch (err) {
      return `계산 오류: ${String(err)}`;
    }
  },
  {
    name: "calculate",
    description: "수학 계산을 수행합니다.",
    schema: z.object({
      expression: z.string(),
    }),
  }
);
// 2. 에이전트 생성
const agent = createAgent({
  model: initChatModel("openai:gpt-4o-mini", {
    temperature: 0.7,
    // apiKey: process.env.OPENAI_API_KEY,  ← 환경변수에서 자동으로 읽음
  })
```

LangChain 1.0 프레임워크 구성

기본 에이전트 예제

```
 tools: [searchWeb, calculate],
  // system prompt
  prompt:
    "당신은 웹 검색과 수학 계산을 도와주는 똑똑한 AI 어시스턴트입니다.₩n" +
    "필요하면 도구를 사용하고, 최종 답변은 명확하고 간결하게 한국어로 작성하세요.",

  // (선택) 최대 반복 횟수 제한
  maxIterations: 12,
});
// 3. 실행 예제
async function run() {
  const result = await agent.invoke({
    messages: [
      {
        role: "user",
        content: "12345 × 6789는 얼마야?",
      },
    ],
  });
  console.log(result.messages.at(-1)?.content);
}
run();
```

C:₩langchain_dev>npx tsx agent-example.ts

LangChain 1.0 프레임워크 구성

기본 에이전트 예제

```
 tools: [searchWeb, calculate],
  // system prompt
  prompt:
    "당신은 웹 검색과 수학 계산을 도와주는 똑똑한 AI 어시스턴트입니다.₩n" +
    "필요하면 도구를 사용하고, 최종 답변은 명확하고 간결하게 한국어로 작성하세요.",

  // (선택) 최대 반복 횟수 제한
  maxIterations: 12,
});
// 3. 실행 예제
async function run() {
  const result = await agent.invoke({
    messages: [
      {
        role: "user",
        content: "12345 × 6789는 얼마야?",
      },
    ],
  });
  console.log(result.messages.at(-1)?.content);
}
run();
```

C:₩langchain_dev>npx tsx agent-example.ts

LangChain 1.0 프레임워크 구성

Middleware

- createAgent()에 전달하는 middleware는 에이전트의 실행 흐름을 가로채서 커스텀 로직을 주입할 수 있게 해주는 확장 지점입니다.
- 에이전트 실행의 각 단계에 훅(hook)을 걸어 다양한 기능을 확장

```
createAgent({
  model,
  tools,
  middleware: [
    logMiddleware,              // 어떤 툴을 불렀는지 로깅
    guardrailMiddleware,        // 위험 키워드 차단
    forceToolMiddleware,        // 특정 상황에선 무조건 특정 툴 사용
  ],
  maxIterations: 12,
});
```

LangChain 1.0 프레임워크 구성

Middleware 역할

- 입력/출력 가드레일 (Guardrails) → 위험한 프롬프트 차단, PII 제거, 금지 키워드 필터링
- 로깅 & 모니터링 강화
- 특정 상황에서 강제 툴 호출
- 컨텍스트/메모리 자동 주입·수정 → 사용자 역할, 세션 정보, 회사 정책 등을 매번 자동 삽입
- 툴 호출 결과 후처리 / 검증 → JSON 파싱 실패 시 재시도, 결과가 비어있으면 다른 툴 강제 호출 등
- 사용자 승인 플로우 (Human-in-the-loop) → 위험도가 높은 툴(이메일 보내기, 결제 등) 호출 전에 중단
- A/B 테스트, 모델 라우팅 → 같은 질문에 대해 다른 모델/프롬프트/툴셋으로 분기

LangChain 이란

Middleware 종류

미들웨어	목적 / 설명
createLoggerMiddleware	상세 실행 로그 (LangSmith 스타일)
createGuardrailMiddleware	입력/출력 콘텐츠 필터링
createForceToolMiddleware	특정 조건에서 무조건 특정 툴 사용
createRateLimitMiddleware	분당/일 호출 수 제한공개 API, 비용 통제 필요 시
createHumanInTheLoopMiddleware	특정 툴/단계에서 인간 승인 대기결제, 계약, 민감 작업
createMemoryInjectionMiddleware	매 턴마다 메모리/컨텍스트 자동 주입 장기 대화, 역할 기반 에이전트커뮤니티
createToolResultValidationMiddleware	Tool 실행 결과를 "검증(Validation)" 하기 위한 미들웨어
createModelFallbackMiddleware	Primary 모델 실패 → Secondary 모델 자동 실행

LangChain 1.0 프레임워크 구성

Log Middleware 적용

```
function createLoggerMiddleware(label = "LOG") {
  return async (state: typeof MessagesAnnotation.State) => {
    const last    = state.messages.at(-1) as BaseMessage;
    const role    = last?.constructor?.name ?? "Unknown";
    const step    = state.messages.length;
    const preview =    typeof last?.content === "string"  ? last.content.slice(0, 80)    :
JSON.stringify(last?.content).slice(0, 80);
    console.log(`[${label}] step=${step} | ${role} | "${preview}"`);
    return {};
  };
}
.......
const tools      = [new Calculator()];
const toolNode   = new ToolNode(tools);
const llm        = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0      });
const llmWithTools = llm.bindTools(tools);

 // 미들웨어 인스턴스 생성
const loggerNode = createLoggerMiddleware("LOGGER");
```

```
C:₩langchain_dev>npx tsx agent-logger.ts
```

LangChain 1.0 프레임워크 구성

GuardrailMiddleware적용

```
function createGuardrailMiddleware(blocklist: string[]) {
  return async (state: typeof MessagesAnnotation.State) => {
    const last = state.messages.at(-1);
    // AIMessage 이고 텍스트 응답인 경우에만 검사
    if (isAIMessage(last) && typeof last.content === "string") {
      for (const word of blocklist) {
        if (last.content.toLowerCase().includes(word.toLowerCase())) {
          console.log(`[GUARDRAIL] 금지 단어 감지: "${word}" → 응답 차단`);
          return {
            messages: [
              new AIMessage("죄송합니다. 해당 내용은 제공해 드릴 수 없습니다."),
            ],
          };
        }
      }
      console.log("[GUARDRAIL] 통과");
    }
    return {};
  };
}
```

```
C:₩langchain_dev>npx tsx agent-guardrail.ts
```

LangChain 1.0 프레임워크 구성

GuardrailMiddleware 적용

```
.........
const llmWithTools = llm.bindTools(tools);

  // 미들웨어 인스턴스 생성 — 금지 단어 목록 전달
  const guardrailNode = createGuardrailMiddleware([
    "폭탄", "해킹", "불법", "마약",
    "bomb", "hack", "illegal", "drug",
  ]);

const callModel = async (state: typeof MessagesAnnotation.State) => {
    const response = await llmWithTools.invoke(state.messages);
    return { messages: [response] };
  };

  const shouldContinue = (state: typeof MessagesAnnotation.State) => {
    const last = state.messages.at(-1);
    if (isAIMessage(last) && last.tool_calls?.length) return "tools";
    return END;
  };
```

LangChain 1.0 프레임워크 구성

GuardrailMiddleware 적용

```
.........
const llmWithTools = llm.bindTools(tools);

  // 미들웨어 인스턴스 생성 — 금지 단어 목록 전달
  const guardrailNode = createGuardrailMiddleware([
    "폭탄", "해킹", "불법", "마약",
    "bomb", "hack", "illegal", "drug",
  ]);

const callModel = async (state: typeof MessagesAnnotation.State) => {
    const response = await llmWithTools.invoke(state.messages);
    return { messages: [response] };
  };

  const shouldContinue = (state: typeof MessagesAnnotation.State) => {
    const last = state.messages.at(-1);
    if (isAIMessage(last) && last.tool_calls?.length) return "tools";
    return END;
  };
```

LangGraph

메모리와 대화 관리

LangGraph

- Node : 하나의 작업 단위(함수)
  입력: state
  출력: state 일부 업데이트

- State : 상태
  모든 노드가 공유
  데이터 흐름의 중심

- Edge : 흐름
  노드 → 노드 연결

메모리와 대화 관리

LangGraph

- Agent의 내부 동작을 직접 설계하는 도구

Chain = 순차 실행만 가능
Agent = 내부 로직이 블랙박스

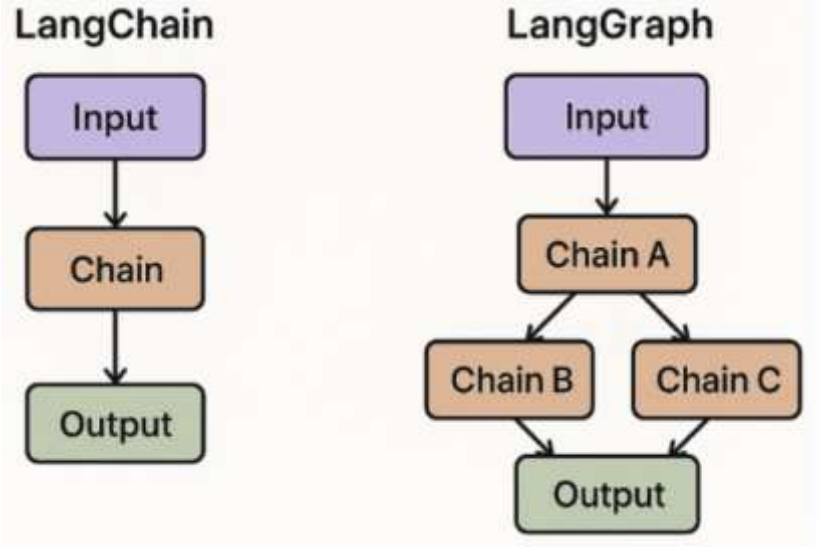
→

조건 분기 어려움
상태 관리 어려움
디버깅 어려움

→

LangGraph = "노드 + 상태 + 흐름 제어"

- Agent 직접 제어하고 싶을 때
- Tool 호출 흐름 제어
- 멀티턴 대화
- 상태 기반 처리
- 복잡한 로직 (조건 분기)



메모리와 대화 관리

LangGraph 기반 메모리

- 상태를 유지하는 그래프 기반의 워크플로우 오케스트레이션 프레임워크
- 복잡한 에이전트(Agent)나 장기 실행이 필요한 LLM 애플리케이션을 구현을 위한 도구
- 상태(State)를 기반으로 흐름을 제어하는 Agent 실행 그래프
- Stateful, controllable, long-running 에이전트 & 워크플로우 구축
- Node(작업) + Edge(흐름) + State(상태) 의 그래프 모델
- .compile() 후 .invoke() / .stream() / .batch() 등을 사용하여 실행
- @langchain/langgraph-checkpoint 패키지로 상태 저장 (Redis, Postgres, Memory 등)
- token-by-token, node 단위, custom 이벤트 스트리밍 모두 지원

간단한 에이전트 → langchain 패키지의 createAgent() 사용 (추천)
복잡하거나 커스텀해야 하는 에이전트 → 직접 @langchain/langgraph 사용
매우 세밀한 제어 또는 multi-agent / hierarchical 구조 → LangGraph 직접 작성

메모리와 대화 관리

LangGraph 기반 메모리

```
import { MemorySaver } from "@langchain/langgraph";
import { createReactAgent } from "@langchain/langgraph/prebuilt";

const checkpointer = new MemorySaver();            // 또는 PostgresSaver, RedisSaver 등

const agent = createReactAgent({
  llm,
  tools,
  checkpointSaver: checkpointer,
  // message_history_key: "messages"  // 기본값
});

const result = await agent.invoke(
  { messages: ["안녕하세요!"] },
  { configurable: { thread_id: "conversation-uuid-999" } }
);
```

메모리와 대화 관리

단기 메모리(Short-term Memory)

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import MemorySaver

# Checkpointer 생성 (메모리 기반)
checkpointer = MemorySaver()

# 에이전트 생성 - checkpointer를 직접 전달
model = init_chat_model("openai:gpt-4o")
agent = create_agent(
    model,
    tools=[],
    checkpointer=checkpointer  # 메모리 활성화
)
# 대화 실행 - agent.invoke() 직접 호출
config = {"configurable": {"thread_id": "conversation-1"}}

result1 = agent.invoke(
    {"messages": [("user", "안녕하세요, 제 이름은 철수입니다.")]},
    config=config
)
```

Appendix

CommonJS(require) vs ES Module

CommonJS(require) vs ES Module

구분	CommonJS (CJS)	ES Module (ESM)
표준	Node.js 전통 방식	ECMAScript 표준
문법	require / module.exports	import / export
로딩 방식	동기 (sync)	비동기 (async)
실행 시점	런타임	정적 분석 (컴파일 단계)

```
// export
module.exports = {
  add: (a, b) => a + b
};

// import
const math = require("./math");
math.add(1, 2);
```

```
// export
export const add = (a, b) => a + b;

// import
import { add } from "./math.js";
add(1, 2);
```

TypeScript

TypeScript

- 타입을 통해 JavaScript를 안전하게 만드는 언어
- 버그 예방, 코드 가독성 , IDE 자동 완성 향상을 목적의 Script

```
//기본타입
let name: string = "홍길동";
let age: number = 30;
let isAdmin: boolean = true;
```

```
//배열
let numbers: number[] = [1, 2, 3];
let names: Array<string> = ["A", "B"];
```

```
//unkown
let value: unknown;

if (typeof value === "string") {
  console.log(value.toUpperCase());
}
```

TypeScript

```
//기본 함수
function add(a: number, b: number): number {
  return a + b;
}
```

```
//화살표 함수
const sub = (a: number, b: number): number => a - b;
```

```
//void
function log(msg: string): void {
  console.log(msg);
}
```

```
//기본 객체
const user: { name: string; age: number } = {
  name: "홍길동",
  age: 30,
};
```

TypeScript

```
//interface 정의
interface User {
  name: string;
  age: number;
}
```

```
//interface 사용
const user: User = {
  name: "홍길동",
  age: 30,
};
```

```
//Type 정의
type User = {
  name: string;
  age: number;
};
```

TypeScript

```
//제네릭 선언
function identity<T>(value: T): T {
  return value;
}
```

```
//제네릭 사용
function identity<T>(value: T): T {
  return value;
}
```

```
//레코드(딕셔너리)
const users: Record<string, number> = {
  a: 1,
  b: 2,
};
```

```
//유니온
let value: string | number;
```

TypeScript

```
//클래스
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }
}
```

```
//접근 제어자
class User {
  public name: string;
  private age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
```

async & await

async & await

- Promise를 "동기 코드처럼" 작성하게 해주는 문법

```
//Callback Hell
getData(function (data) {
  process(data, function (result) {
    save(result, function () {
      console.log("끝");
    });
  });
});
```

```
//Promise 체인
getData()
  .then(data => process(data))
  .then(result => save(result))
  .then(() => console.log("끝"));
```

```
//async : "이 함수는 Promise를 반환한다"
//await : Promise가 끝날 때까지 기다렸다가 결과 반환
async function run() {
  const data = await getData();
  const result = await process(data);
  await save(result);

  console.log("끝");
}
```